Hochschule Karlsruhe
University of
Applied Sciences

Fakultät für
**Maschinenbau und
Mechatronik**

Implementierung von Last-Mile-Delivery-Robotern in CARLA

F&E-Projektarbeit

Anvin Le, Ricardo Rauch
Matrikelnummer: 106539, 73743

am
Institut für Energieeffiziente Mobilität
Hochschule Karlsruhe

Betreuender Professor:	Prof. Dr.-Ing. Reiner Kriesten
Projektkoordinator:	Prof. Dr. rer. nat. Andreas Helfrich-Schkarbanenko
Betreuer am Institut:	Sebastian Gahm M.Sc.

Karlsruhe, den 28.10.2026

Kurzfassung

Die Arbeit bildet zwei Last-Mile-Delivery-Roboter in der Simulationsumgebung CARLA ab: den Gehwegroboter Starship und den Straßenroboter Clevon 1. Beide Fahrzeuge wurden in Blender modelliert, in Unreal Engine 5.5.4 mit Chaos Vehicles aufgebaut und unter CARLA 0.10.0 als fahrfähige Fahrzeuge registriert. Der Starship besitzt sechs angetriebene Räder und einen Bordstein-Assistenten als C++-Komponente, der Clevon ein 6-kW-Antriebsmodell, Lichtfunktionen und eine bewegliche Ladeklappe. Die Sensorik umfasst beim Starship 26 und beim Clevon elf virtuelle Sensoren, darunter Kameras, Radar, GNSS und IMU; ihre Parameter sind aus Herstellerangaben, Patenten und Referenzsensoren abgeleitet und in JSON-Profilen hinterlegt. Über die native ROS-2-Anbindung von CARLA und einen Ergänzungsknoten stehen Sensordaten, Odometrie und Koordinatensysteme unter festen Topic-Namen bereit; Fahrbefehle können über ROS 2 gesendet werden. Funktionstests belegen Fahrbarkeit, Datenempfang und Steuerung. Grenzen bestehen in fehlenden Vergleichsmessungen an den realen Fahrzeugen, idealisierten Sensoren und der vereinfachten Lenk- und Kletterkinematik des Starship. Die Modelle bilden eine Grundlage für die Anbindung externer Perception-Module und für Untersuchungen zur Sicherheit vernetzter Lieferroboter.

Abstract

This report models two last-mile delivery robots in the CARLA simulator: the Starship sidewalk robot and the Clevon 1 road robot. Both vehicles were modelled in Blender, built in Unreal Engine 5.5.4 with Chaos Vehicles and registered as drivable vehicles in CARLA 0.10.0. The Starship has six driven wheels and a curb-climbing assist implemented as a C++ component; the Clevon has a 6 kW drive model, lighting functions and a movable cargo door. The sensor setup comprises 26 virtual sensors for the Starship and eleven for the Clevon, including cameras, radar, GNSS and IMU. Their parameters are derived from manufacturer data, patents and reference sensors and are stored in JSON profiles. CARLA's native ROS 2 interface and a supplementary node provide sensor data, odometry and coordinate frames under fixed topic names, and driving commands can be sent via ROS 2. Functional tests confirm drivability, data reception and control. Limitations are the lack of reference measurements on the real vehicles, idealised sensors and the simplified steering and climbing kinematics of the Starship. The models provide a basis for connecting external perception modules and for security studies of connected delivery robots.

Eidesstattliche Erklärung

Hiermit versichern wir, dass wir die vorliegende Projektarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen oder anderen Quellen entnommen sind, sind als solche kenntlich gemacht. Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegt.

Karlsruhe, den 28.10.2026
Anvin Le

Karlsruhe, den 28.10.2026
Ricardo Rauch

Erklärung zur Nutzung von KI-Werkzeugen

Bei der Erstellung dieser Arbeit wurden KI-basierte Werkzeuge (u. a. Claude von Anthropic und ChatGPT/Codex von OpenAI) in folgenden Bereichen eingesetzt:

- **Sprachliche Glättung:** Überarbeitung von Formulierungen, Rechtschreibung und Struktur selbst verfasster Texte.
- **Sparringspartner und Brainstorming:** Diskussion von Lösungsansätzen, Gliederungen und Fehlerursachen.
- **Programmierung:** Unterstützung beim Erstellen und Prüfen von Python-, C++- und Unreal-Editor-Skripten.

Inhalte, Messwerte und Quellenangaben wurden von den Verfassern geprüft. Alle Messungen beruhen auf eigenen Versuchen in der Simulation. Die Verantwortung für den Inhalt der Arbeit liegt vollständig bei den Verfassern.

Inhaltsverzeichnis

Kurzfassung i

Abstract ii

Eidesstattliche Erklärung iii

Erklärung zur Nutzung von KI-Werkzeugen iv

Abbildungsverzeichnis ix

Tabellenverzeichnis x

1 Einleitung und Zielsetzung 1
1.1 Motivation . . . 1
1.2 Aufgabenstellung und Fragestellung . . . 1
1.3 Abgrenzung . . . 1

2 Anforderungen und Bewertungsmaßstab 2

3 Grundlagen und Entwicklungsumgebung 3
3.1 Modellierung, Simulation und Softwareversion . . . 3
3.2 Sensorprinzipien . . . 3
3.2.1 ROS 2 und Simulationszeit . . . 4

4 Umsetzungskonzept 5

5 Implementierung des Starship-Gehwegroboters 7
5.1 Referenzfahrzeug und Modellumfang . . . 7
5.1.1 Eigene Modellierung und Herkunft der Bauteile . . . 7
5.2 Geometrie, Rig und CARLA-Integration . . . 7
5.2.1 Abmessungen . . . 8
5.2.2 Aufgaben der Blueprint-Bausteine . . . 9
5.2.3 Skelettanimation und Kollisionskörper . . . 9
5.2.4 Skriptgestützter Aufbau . . . 10
5.3 Fahrphysik und Antrieb . . . 11
5.3.1 Masse, Schwerpunkt und Fahrwerk . . . 11
5.3.2 Allradantrieb . . . 12
5.3.3 Bordstein-Assistent . . . 12
5.4 Sensorik und Parametrierung . . . 13
5.4.1 Kameras und Blickrichtungen . . . 14
5.4.2 ToF-Sensoren und Ultraschall . . . 15
5.4.3 Radar . . . 15
5.4.4 GNSS, IMU und Messraten . . . 16
5.5 Machbarkeit weiterer Funktionen . . . 16

6 Implementierung des Clevon 1 **17**
6.1 Referenzfahrzeug und Modellumfang . . . 17
6.1.1 Eigene Modellierung und Herkunft der Bauteile . . . 17
6.2 Geometrie, Rig und CARLA-Integration . . . 18
6.2.1 Abmessungen . . . 19
6.2.2 Aufgaben der Blueprint-Bausteine . . . 20
6.2.3 Skelettanimation und Kollisionskörper . . . 20
6.2.4 Event Graph und Aktualisierungsablauf . . . 21
6.2.5 Bremssättel: Federbewegung und Lenkung . . . 22
6.3 Fahrphysik und Antrieb . . . 24
6.3.1 Masse, Schwerpunkt und Fahrwerk . . . 24
6.3.2 Aggregiertes 6-kW-Modell . . . 25
6.4 Sensorik und Parametrierung . . . 25
6.4.1 Kameras und Blickrichtungen . . . 27
6.4.2 Radar und ergänzender LiDAR . . . 27
6.4.3 GNSS, IMU und Messraten . . . 28
6.5 Lichtfunktionen und Ladeklappe . . . 28
6.5.1 Leuchtflächen und Ausleuchtung . . . 28
6.5.2 Verarbeitung der CARLA-Lichtzustände . . . 28
6.5.3 Blinker und eingestellte Lichtintensitäten . . . 29
6.5.4 Ladeklappe . . . 30

7 ROS-2-Anbindung **31**
7.1 Architektur . . . 31
7.2 Verknüpfung mit den Sensorskripten . . . 31
7.3 Namensgebung und Koordinatensysteme . . . 33
7.4 Ergänzungsknoten . . . 33
7.5 Fahrzeugsteuerung über ROS 2 . . . 33
7.6 Betrieb . . . 34
7.7 Übertragungsraten . . . 34
7.8 Grenzen . . . 35

8 Validierung und Ergebnisse **36**
8.1 Prüfumfang . . . 36
8.2 Geometrie . . . 36
8.3 Fahrphysik . . . 36
8.4 Sensorik und ROS 2 . . . 36
8.5 Zusammenfassung und Grenzen . . . 37

9 Diskussion und Grenzen **38**
9.1 Gemeinsame Grenzen . . . 38
9.2 Starship . . . 38
9.3 Clevon . . . 39
9.4 Vergleich der Fahrzeugtypen . . . 39

10 Fazit und Ausblick **40**

Literaturverzeichnis **41**

A Clevon: Sensorzeichnungen **44**

B Reproduzierbarkeit **47**

Abbildungsverzeichnis

5.1 Starship-Modell im HKA-Design mit sechs Rädern und Antenne. Eigene Darstellung aus Blender. 8
5.2 Komponentenstruktur des Starship-Fahrzeug-Blueprints in Unreal Engine 5.5.4. Eigener Screenshot. 9
5.3 Radanimation und Kollisionskörper des Starship in Unreal Engine 5.5.4. Eigene Screenshots. 10
5.4 Auffahren auf einen 13,6 cm hohen Bordstein in Town10HD mit Bordstein-Assistent. Eigene Aufnahmen aus CARLA 0.10.0. 13
5.5 Sensoranordnung am Starship-Modell: Kameras (K), ToF (T), Radar (R), Ultraschall (U) und GNSS (G); Kegel zeigen die Blickrichtung. Eigene Darstellung. 14
5.6 Anzeigeprogramme der Starship-Sensorik in CARLA 0.10.0. Eigene Screenshots. 15

6.1 Bildgestützte Rekonstruktion der Clevon-Karosserie (hier noch ohne LiDAR). Referenzbild: Clevon (2023, S. 7); Modellansicht: eigene Darstellung. 17
6.2 Clevon-Modell mit CargoBox und ergänzendem Dach-LiDAR. Eigene Darstellung aus Blender; neutrale Materialansicht. 18
6.3 Anordnung des Basisknochens `Vehicle_Base` und der vier Radknochen im Clevon-Modell. Eigene Darstellung aus Blender. 19
6.4 Komponentenstruktur des Clevon-Fahrzeug-Blueprints in Unreal Engine 5.5.4. Eigener Screenshot. 20
6.5 Radanimation und Kollisionskörper des Clevon in Unreal Engine 5.5.4. Die dargestellten Radkörper sind von den Spherecast-Abfragen für den Rad-Boden-Kontakt zu unterscheiden. Eigene Screenshots. . . . 21
6.6 Ereignisse und Funktionsaufrufe im Event Graph von `BP_Clevon1` in Unreal Engine 5.5.4. Eigene Screenshots. 22
6.7 Getrennte Aktualisierung von Position und Lenkung der Bremssättel in Unreal Engine 5.5.4. Die Rollrotation der Radknochen wird nicht übernommen. Eigene Screenshots. 23
6.8 Drehmoment und daraus berechnete mechanische Leistung des 6-kW-Ersatzmodells. Eigene Berechnung aus den gespeicherten Stützstellen. 26
6.9 Anzeigeprogramme der Clevon-Sensorik in CARLA 0.10.0. Eigene Screenshots. 27
6.10 Auschnitt Ansteuerung von Leuchtmaterial und Lichtquelle aus dem CARLA-Lichtzustand. Eigener Screenshot aus Unreal. 29
6.11 Lichtfunktionen am integrierten Clevon in CARLA. Eigene Aufnahme. 29
6.12 Modellansicht des Clevon mit geöffneter CargoBox-Ladeklappe. Eigene Darstellung. 30

7.1 Architektur des Aufbaus. Python-Clients steuern CARLA über die CARLA-Python-API (TCP) und zeigen Sensordaten in pygame-Fenstern an. Mit `--ros2` veröffentlicht CARLA die Sensordaten selbst über DDS; der Ergänzungsknoten liefert Fahrzeuglage und Ultraschall. Fahrbefehle erreichen CARLA über `vehicle_control_cmd`. Eigene Darstellung. 32

Tabellenverzeichnis

5.1 Veröffentlichte Abmessungen (Dimensions.com, n. d.) und eigene Geometriemessung in CARLA. 8
5.2 Wesentliche Geometrie- und Fahrwerksparameter des Starship-Modells unter Chaos Vehicles. Die Einstellungen stellen keine Herstellerkalibrierung dar. 11
5.3 Sensorprofile des Starship-Modells. Messraten sind Sollwerte in Simulationszeit. 14

6.1 Abmessungen gemäß Herstellerproduktblatt (Clevon, n. d. b, S. 2) und eigener Geometriemessung. 19
6.2 Wesentliche Geometrie- und Fahrwerksparameter des Clevon-Modells unter Chaos Vehicles. Die Einstellungen stellen keine vollständige OEM-Kalibrierung dar. 24
6.3 Sensorprofile des Clevon-Modells. Messraten sind Sollwerte in Simulationszeit; beim LiDAR ist die Rotationsfrequenz angegeben. 26

7.1 Sensorgruppen und Profile je Fahrzeug 32
7.2 Steuerung über ROS 2 (Town10HD, ohne Fahrclient) 34
7.3 Gemessene Übertragungsraten (Town10HD, asynchron, alle Sensoren aktiv) 35

8.1 Fahrphysikalische Ergebnisse in Town10HD; eigene Messung bzw. Beobachtung. 36
8.2 Nachweise je Prüfbereich. 37

1 Einleitung und Zielsetzung

1.1 Motivation

Last-Mile-Delivery-Roboter stellen eine aufkommende Anwendung vernetzter autonomer Fahrzeugsysteme dar. Sie unterscheiden sich hinsichtlich ihrer Bauweise, ihres Einsatzbereichs und ihrer technischen Ausstattung. Ein Beispiel für den praktischen Einsatz ist die Clevon-DHL-Fallstudie, in der ein elektrischer Lieferroboter für Zustellungen in Tallinn eingesetzt wurde (Clevon, 2023).

Mit der zunehmenden Vernetzung und Automatisierung dieser Systeme entstehen neue Anforderungen an die Untersuchung ihrer Funktionalität sowie möglicher Sicherheitsrisiken, insbesondere im Bereich der IT-Sicherheit (Security). Simulationsumgebungen bieten die Möglichkeit, solche Systeme unter kontrollierten und reproduzierbaren Bedingungen zu untersuchen.

Ziel dieser Arbeit ist es daher, zwei unterschiedliche Last-Mile-Delivery-Roboter möglichst originalgetreu in der Simulationsumgebung CARLA abzubilden. Neben der Fahrzeugmodellierung und Sensorintegration soll insbesondere die Anbindung externer Perception-Modelle über eine ROS-2-Schnittstelle ermöglicht werden. Damit schafft die Arbeit die technische Grundlage für weiterführende Untersuchungen, insbesondere zur Analyse möglicher Sicherheitsrisiken vernetzter autonomer Systeme.

1.2 Aufgabenstellung und Fragestellung

Die Projektaufgabe umfasst einen Gehwegroboter und einen Straßenroboter mit parametrierter Fahrphysik und Sensorik sowie einer dokumentierten ROS-2-Anbindung. Zusätzlich sind Funktionstests in CARLA gefordert (Hochschule Karlsruhe, Institut für Energieeffiziente Mobilität, 2026). Die leitende Frage lautet: Wie lassen sich zwei unterschiedliche Lieferroboter als fahrfähige, sensorisch ausgestattete CARLA-Modelle umsetzen und für externe ROS-2-Module bereitstellen?

1.3 Abgrenzung

Gegenstand sind Modellgeometrie, Fahrzeugfunktionen, Simulationsparameter und Daten- sowie ggf. Befehlsschnittstellen. Autonome Wahrnehmung, Routenplanung und Fahrentscheidungen werden nicht entwickelt. Fehlende Herstellerparameter werden als Annahmen bzw. Best Practice ausgewiesen. Funktionstests der Simulation ersetzen keine messtechnische Validierung an den realen Fahrzeugen.

2 Anforderungen und Bewertungsmaßstab

Aus der Aufgabenstellung werden vier Prüfbereiche abgeleitet (Hochschule Karlsruhe, Institut für Energieeffiziente Mobilität, 2026):

1. Geometrie: nachvollziehbare Abmessungen und korrekt skalierter Import.
2. Fahrphysik: angesetzte Masse, Rad-/Fahrwerksparameter und plausibles Antriebsverhalten.
3. Sensorik: definierte Montage, Messparameter und nachgewiesener Datenempfang.
4. ROS 2: dokumentierte Datenformate, Koordinaten, Zeitstempel und ausführbare Fahrzeugbefehle.

Die Bewertung unterscheidet Herstellerangaben, berechnete Modellparameter und experimentell beobachtete Eigenschaften. Numerische Messungen und qualitative Sichtprüfungen werden getrennt angegeben. Für den Clevon wird der betrachtete Sensorbestand mit Kameras, Radar, GNSS und IMU abgebildet; LiDAR ist eine zusätzliche Variante. Ultraschall wird bei diesem Fahrzeug nicht ergänzt.

Für den Starship werden Kameras, Radar, Ultraschall, ToF-Tiefensensoren, GNSS und IMU abgebildet; Ultraschall und ToF werden in CARLA nachgebildet. LiDAR wird bei diesem Fahrzeug nicht ergänzt. Damit wird die allgemeine Sensorliste der Aufgabenstellung fahrzeugspezifisch eingegrenzt.

3 Grundlagen und Entwicklungsumgebung

3.1 Modellierung, Simulation und Softwareversion

Blender dient zur Modellierung der Fahrzeuggeometrie und des Skeletts. Unreal Engine verbindet diese mit Materialien, Kollisionskörpern, Animationen und Fahrzeugfunktionen. CARLA ermöglicht den Zugriff auf Simulation und virtuelle Sensoren über seine Python-API (CARLA Team, n. d. c).

Beide Fahrzeugmodelle wurden zunächst für CARLA 0.9.16 mit Unreal Engine 4.26.2 entwickelt und anschließend auf CARLA 0.10.0 mit Unreal Engine 5.5.4 migriert. Gründe für den Umstieg sind die aktuelle Engine-Generation mit verbesserter Darstellung von Beleuchtung und Umgebung, die Chaos-Fahrzeugphysik mit beliebig vielen Rädern sowie die in CARLA 0.10 integrierte ROS-2-Schnittstelle, die Sensordaten ohne separate Bridge veröffentlicht. Zugleich werden beide Fahrzeuge damit auf eine gemeinsame Plattform zusammengeführt.

Mit der Migration ersetzte Chaos Vehicles die bisherige PhysX-Fahrzeugphysik. Beim Clevon konnten Geometrie, Materialien, Skelett und wesentliche Blueprint-Funktionen übernommen werden; Radklassen, Animationen und physikabhängige Parameter wurden auf Chaos angepasst (Kapitel 6). Die Unreal-Engine-4-Assets des Starship ließen sich nicht übernehmen; er wurde aus dem exportierten Modell und den dokumentierten Fahrzeugwerten neu aufgebaut (Kapitel 5).

Die gemeinsame Zielplattform umfasst CARLA 0.10.0, Unreal Engine 5.5.4 und ROS 2 Jazzy unter Ubuntu 24.04.2 LTS. Während der Clevon ursprünglich mit ROS 2 Humble unter Ubuntu 22.04/WSL 2 geprüft wurde, beziehen sich die beschriebenen ROS-2-Tests der neuen Plattform auf die native CARLA-Anbindung unter Linux.

3.2 Sensorprinzipien

RGB-Kameras liefern Bilddaten; für ihre geometrische Interpretation sind Auflösung, Sichtfeld sowie Position und Orientierung der Kamera relevant (CARLA Team, n. d. a, n. d. e). LiDAR liefert eine räumliche Punktwolke. Der CARLA-Radar liefert Entfernung, Winkel und radiale Relativgeschwindigkeit geometrischer Treffer. GNSS gibt geografische Koordinaten aus, während die IMU lineare Beschleunigung und Winkelgeschwindigkeit erfasst (CARLA Team, n. d. e). Für die ToF- und Ultraschallsensoren des Starship stellt CARLA keine eigenen Sensortypen bereit; sie werden durch verwandte Sensoren nachgebildet (CARLA Team, n. d. e). Die Tiefenkamera liefert je Bildpunkt die Entfernung zur Szene und ersetzt damit den ToF-Sensor. Der Hinderniserkennungssensor ersetzt den Ultraschall. Er prüft entlang einer Linie vor dem Sensor, ob ein Objekt getroffen wird, und gibt dessen Entfernung aus. Schallkeule, Reflexionen und materialabhängige Echos werden dabei nicht abgebildet (CARLA Team, n. d. e). Die dokumentierten Rausch- und Ausfallmodelle bilden ausgewählte Sensoreigenschaften ab; daraus lässt sich keine vollständige Abbildung aller Störungen realer Hardware ableiten (CARLA Team, n. d. e).

3.2.1 ROS 2 und Simulationszeit

ROS 2 tauscht Daten nach dem Publish-Subscribe-Prinzip über benannte Datenkanäle (Topics) aus; die Übertragung übernimmt die Middleware DDS (Open Robotics, n. d. c). Über Quality-of-Service-Einstellungen wird festgelegt, ob Nachrichten zuverlässig zugestellt oder bei Verlust verworfen werden (*reliable* bzw. *best effort*) (Open Robotics, n. d. a). TF beschreibt die räumlichen Beziehungen zwischen Koordinatensystemen, hier zwischen Karte, Fahrzeug und Sensoren (Open Robotics, n. d. b).

CARLA 0.10 veröffentlicht die Sensordaten direkt aus dem Simulator (CARLA Team, n. d. d). Jeder Nachricht wird die Simulationszeit als Zeitstempel mitgegeben, die zusätzlich auf `/clock` bereitsteht.

Für die Übertragungsrate ist der Takt der Simulation maßgeblich. Ein Sensor erzeugt höchstens eine Messung je Simulationsschritt; das Attribut `sensor_tick` legt nur den Mindestabstand zwischen zwei Messungen fest (CARLA Team, n. d. e). Die erreichbare Rate ist daher das Minimum aus konfigurierter Sensorrate und Simulationstakt. Im asynchronen Betrieb läuft die Simulation so schnell, wie es die Rechen- und Renderlast zulässt. Zusätzliche Sensoren, insbesondere Kameras mit hoher Auflösung, senken damit den Simulationstakt und zugleich die Rate aller übrigen Sensoren. Der synchrone Betrieb mit festem Zeitschritt liefert dagegen für jeden Simulationsschritt vollständige Messungen, erfordert aber die ausdrückliche Steuerung der Schritte durch einen Client und läuft bei hoher Last langsamer als Echtzeit (CARLA Team, n. d. f).

4 Umsetzungskonzept

Beide Fahrzeuge werden nach derselben Prozesskette aufgebaut: Referenzdaten, Geometrie, Rig und Export, Integration in Unreal Engine und CARLA, Parametrierung, Sensorik und Funktionsprüfung. Die Kette folgt der CARLA-Anleitung zum Hinzufügen eigener Fahrzeuge (CARLA Team, n. d. b) und wurde durch Video-Tutorials zur Fahrzeugmodellierung in Blender ergänzt (Gallant, n. d.; Gryd CG, n. d.).

1. **Referenzdaten:** Abmessungen, Massen und Sensoranordnung aus Herstellerangaben, Patenten und Veröffentlichungen; fehlende Werte werden begründet angenommen und als solche gekennzeichnet.

2. **Geometrie und Rig:** Modellierung in Blender mit getrennten Radobjekten. Ein Skelett mit Wurzelknochen und je einem Knochen pro Rad verknüpft Karosserie und Räder; Export als FBX in Zentimetern.

3. **Integration:** Import als Skeletal Mesh in Unreal Engine, Kollisionskörper im Physics Asset, Radanimation über ein Animation Blueprint, Radklassen und Fahrzeug-Blueprint auf Basis von Chaos Vehicles, Registrierung als CARLA-Fahrzeug.

4. **Parametrierung:** Masse, Schwerpunkt, Federung, Antrieb und Bremsen werden im Blueprint hinterlegt und über die Python-API ausgelesen und geprüft.

5. **Sensorik:** Sichtbare Sensorgehäuse gehören zum 3D-Modell; die Messfunktion entsteht durch CARLA-Sensoren, die Python-Skripte zur Laufzeit am Fahrzeug anbringen.

6. **Funktionsprüfung:** Skriptgesteuerte Fahr- und Sensortests in einer Referenzkarte mit gespeicherten Messdaten.

Sensorpositionen und -attribute liegen je Sensortyp in JSON-Profilen. Anzeigeprogramme und ROS-2-Anbindung lesen dieselben Profile, sodass eine Änderung an einer Stelle für alle Anwendungen gilt. Gemeinsame Funktionen wie Verbindungsaufbau, Profilprüfung und Sensorverwaltung sind je Fahrzeug in einem Modul mit gleicher Schnittstelle gebündelt.

Die Umsetzung erfolgt iterativ: Abweichungen im Fahrverhalten werden gemessen, ihre Ursache im Quellcode von Unreal Engine und CARLA nachvollzogen und erst danach Parameter oder Komponenten geändert. Jeder Schritt ist in einem Implementierungsprotokoll mit Messwerten festgehalten; der Aufbau in Unreal Engine ist, wo möglich, über Skripte reproduzierbar. Modellquellen, Unreal-Assets, Skripte und ausgewählte Prüfdaten liegen in einem Git-Repository (Rauch & Le, 2026).

Fahrzeuglokale CARLA-Koordinaten verwenden ein linkshändiges System mit x nach vorne, y nach rechts und z nach oben. ROS verwendet nach REP 103 ein rechtshändiges System mit y nach links und SI-Einheiten (Foote & Purvis, n. d.); Koordinatensysteme werden nach REP 105 benannt (Meeussen, n. d.). Positionen werden durch $(x, y, z) \mapsto (x, -y, z)$, Rotationen durch $(\varphi, \theta, \psi) \mapsto (\varphi, -\theta, -\psi)$ überführt. Die Montageprofile bleiben in CARLA-Koordinaten; umgerechnet wird erst bei der Veröffentlichung in ROS 2.

Die fahrzeugspezifische Umsetzung von Clevon und Starship wird in den folgenden Kapiteln beschrieben.

5 Implementierung des Starship-Gehwegroboters

Dieses Kapitel beschreibt die Modellierung und Integration des Starship-Gehwegroboters in CARLA. Im Mittelpunkt stehen die Fahrzeuggeometrie, die Umsetzung der Fahrphysik mit sechs simulierten Rädern unter Chaos Vehicles, ein Bordstein-Assistent sowie die Sensorik. Beschrieben wird der abschließende Implementierungsstand unter CARLA 0.10.0 und Unreal Engine 5.5.4; die ROS-2-Anbindung folgt in Kapitel 7.

5.1 Referenzfahrzeug und Modellumfang

Der Starship-Roboter bildet den gehwegbasierten Lieferroboter des Projekts ab. Es handelt sich um ein sechsrädriges Fahrzeug, das mit Schrittgeschwindigkeit auf Gehwegen fährt. Eine technische Übersicht nennt eine Höchstgeschwindigkeit von 6 km/h, eine Gesamtmasse von 35 kg und eine Zuladung von 10 kg (Wevolver, n. d.). Ein Patent des Herstellers beschreibt das Überwinden von Bordsteinen mit etwa 15 bis 20 cm Höhe über Kipphebel zwischen Mittel- und Hinterrädern und nennt einen bevorzugten Raddurchmesser von etwa 20 cm (Liivik et al., 2024). Herstellerdatenblätter liegen nicht vor; die Angaben stammen aus veröffentlichten Übersichten und Patenten. Sie dienen als Referenz für die Modellierung und Parametrierung.

Das Simulationsmodell umfasst die Fahrzeuggeometrie, Fahr- und Lenkfunktionen mit Antrieb an allen sechs Rädern, einen Bordstein-Assistenten sowie virtuelle Sensoren. Lichtfunktionen und eine bewegliche Ladeklappe sind nicht Bestandteil des Modells; der Deckel ist Teil der starren Karosserie.

5.1.1 Eigene Modellierung und Herkunft der Bauteile

Karosserie, Räder und Antenne wurden in Blender eigenständig modelliert. Das Modell übernimmt die Grundform des Originals mit kastenförmigem Aufbau, Deckel, Antenne und drei Rädern je Seite, ist aber in einem eigenen HKA-Design ausgeführt: anthrazitfarbener Korpus, roter Schriftzug „HKA“ auf beiden Seiten und ein rotes Licht an der Antenne anstelle der Fahne des Originals (Abbildung 5.1). Die Geometrie ist damit eine vereinfachte Nachbildung und kein übernommenes CAD-Modell. Alle Bauteile wurden selbst erstellt; Fremdmodelle wurden nicht verwendet.

Die sichtbaren Sensorgehäuse sind nicht einzeln modelliert. Die Messfunktion wird ausschließlich durch die zugeordneten virtuellen Sensoren in CARLA bereitgestellt (Abschnitt 5.4).

5.2 Geometrie, Rig und CARLA-Integration

Das Fahrzeugskelett besteht aus dem Basisknochen `VehicleBase` und sechs Radknochen `Wheel_{Front,Middle,Rear}_{Left,Right}`, die jeweils im Radmittelpunkt liegen. Die Radknochen liegen 9 cm über dem Boden bei $x = +19{,}1$, 0,1 und $-19{,}1$ cm



Abbildung 5.1: Starship-Modell im HKA-Design mit sechs Rädern und Antenne. Eigene Darstellung aus Blender.

sowie $y = \pm 25{,}6\,\text{cm}$. Daraus ergeben sich ein Radstand von 38,2 cm zwischen Vorder- und Hinterachse und eine Spurweite von 51,2 cm. Die Radgeometrien sind starr an die Knochen gebunden.

Alle Knochen sind in Blender mit der Spitze in $+y$-Richtung und ohne Rollwinkel angelegt. Ein gekippter Basisknochen führt in Unreal Engine zu einem gedrehten Fahrzeugbezugssystem: Die Federungsabfragen zeigen dann zur Seite, alle Räder gelten als in der Luft, und das Fahrzeug fährt nicht. Der FBX-Export verwendet Zentimeter; der Importfaktor beträgt 1, Mesh und Skelett sind nicht skaliert. Die Modellquellen `starship_hka_v5.blend` und `starship_hka_v5.fbx` liegen im Repository unter `02_starship_emodell/models` (Rauch & Le, 2026).

Da Unreal-Engine-4-Assets in Unreal Engine 5 nicht geladen werden können, wurde das Fahrzeug unter CARLA 0.10.0 aus der FBX-Datei und den dokumentierten Fahrzeugwerten neu aufgebaut. Der Fahrzeug-Blueprint `BP_Starship` basiert auf `BaseVehiclePawnNW`, der Basisklasse von CARLA für Fahrzeuge mit mehr als vier Rädern. Seine Registrierung erfolgt über die `VehicleParameters.json` unter der Kennung `vehicle.starship.bot` mit sechs Rädern.

5.2.1 Abmessungen

Tabelle 5.1 vergleicht die veröffentlichten Abmessungen mit den Außenmaßen des Modells. Länge und Breite entsprechen der von CARLA gemeldeten Actor-Bounding-Box von $0{,}565 \times 0{,}554 \times 0{,}657\,\text{m}$; diese schließt die Antenne ein. Die Höhe ohne Antenne wurde in CARLA per Strahlabfrage auf den Deckel bestimmt.

Tabelle 5.1: Veröffentlichte Abmessungen (Dimensions.com, n. d.) und eigene Geometriemessung in CARLA.

Größe	Referenz	Modell	Abweichung
Länge	0,678 m	0,565 m	−16,7 %
Breite	0,569 m	0,554 m	−2,6 %
Höhe ohne Antenne	0,554 m	0,53 m	−4,3 %
Höhe mit Antenne	–	0,657 m	–
Raddurchmesser	ca. 0,20 m	0,18 m	−10 %

Die Abweichungen kennzeichnen das eigene Design und die vereinfachte Nach-

bildung. Die Länge weicht am stärksten ab, da Front und Heck des Modells kürzer ausgeführt sind. Die Referenzangaben stammen aus einer frühen Fahrzeuggeneration; eine zweite Übersicht nennt 697 × 569 × 571 mm ohne Fahne (Wevolver, n. d.). Der Raddurchmesser folgt dem Patent (Liivik et al., 2024).

5.2.2 Aufgaben der Blueprint-Bausteine

Das Fahrzeug entsteht in Unreal Engine aus dem Zusammenspiel mehrerer Assets. Das Skeletal Mesh `SK_Starship` enthält die sichtbare Geometrie und das Fahrzeugskelett. Das Physics Asset beschreibt den physikalischen Körper. Sechs Chaos-Wheel-Blueprints `BP_Starship_{FLW,FRW,MLW,MRW,RLW,RRW}` legen die radbezogenen Parameter fest, darunter Radius, Lenkfähigkeit, Antrieb, Bremsmomente und Federung. Die Chaos-Fahrzeugbewegungskomponente berechnet die Fahrzeugbewegung; der Animations-Blueprint `AnimBP_Starship` überträgt die abgeleiteten Radbewegungen auf die Radknochen. Der Fahrzeug-Blueprint `BP_Starship` verbindet diese Bausteine.

Die Komponentenstruktur umfasst neben dem Fahrzeug-Mesh die von `BaseVehiclePawnNW` geerbten Komponenten sowie die zusätzliche Komponente `CurbAssist` für den Bordstein-Assistenten (Abbildung 5.2). Separate Glas- oder Leuchtenmeshes und Lichtquellen werden nicht benötigt. Die geerbte Komponente `VehicleBounds` passt CARLA beim Erzeugen selbst an das Mesh an.

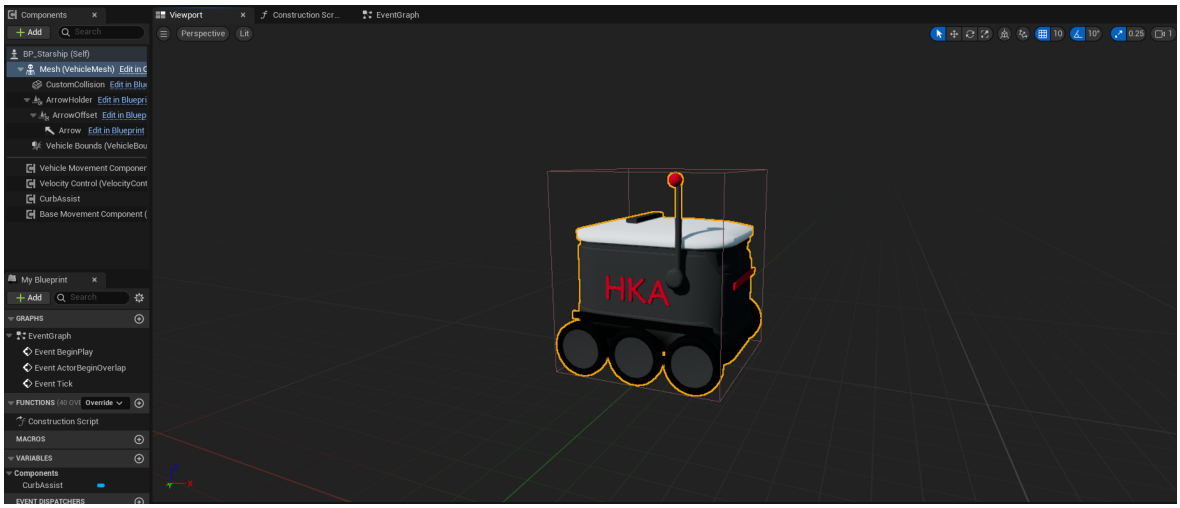
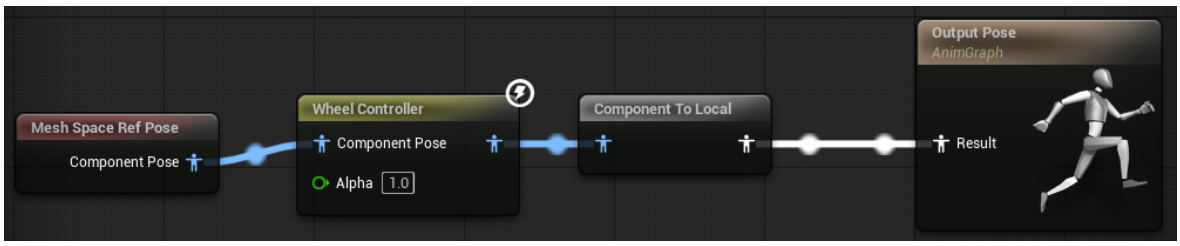


Abbildung 5.2: Komponentenstruktur des Starship-Fahrzeug-Blueprints in Unreal Engine 5.5.4. Eigener Screenshot.

5.2.3 Skelettanimation und Kollisionskörper

Der Animations-Blueprint `AnimBP_Starship` ist eine Kopie des sechsrädrigen Referenzfahrzeugs `EuropeanHGV` aus CARLA, umgestellt auf das Starship-Skelett. Der Animationsgraph verbindet *Mesh Space Ref Pose*, *Wheel Controller*, *Component To Local* und *Output Pose* (Abbildung 5.3). Der *Wheel Controller* erhält Radknochen, Raddrehung, Lenkwinkel und Federweg zur Laufzeit aus den Radkonfigurationen der Bewegungskomponente und ist daher nicht an die Knochennamen des Referenzfahrzeugs gebunden. Da alle sechs Räder simuliert werden, entfällt die frühere Lösung unter Unreal Engine 4, bei der die Mittelräder die Drehung der Hinterräder nur optisch kopierten.

Das Physics Asset enthält genau einen Kollisionskörper: eine Box von $55 \times 45 \times 30$ cm auf dem Basisknochen. Ihre Unterkante liegt 19 cm, ihre Oberkante 49 cm über dem Boden. Die Radknochen besitzen keine eigenen Körper. Die Rad-Boden-Kontakte werden durch die Spherecast-Abfragen der Chaos-Fahrzeugbewegungskomponente ermittelt. Die Unterkante der Box liegt damit 4 cm über einem 15 cm hohen Bordstein, auch wenn die Vorderräder bereits oben sind und der Aufbau noch nicht.



(a) Chaos-Animationsgraph zur Übertragung der simulierten Radbewegungen auf das Fahrzeugskelett.



(b) Physics Asset mit einem Kollisionskörper auf `VehicleBase`. Die Räder besitzen keine eigenen Körper.

Abbildung 5.3: Radanimation und Kollisionskörper des Starship in Unreal Engine 5.5.4. Eigene Screenshots.

5.2.4 Skriptgestützter Aufbau

Der Aufbau in Unreal Engine ist vollständig über Python-Skripte reproduzierbar, die über die Remote-Ausführung des Editors laufen (Rauch & Le, 2026). Die Skripte importieren das Mesh, erzeugen das Physics Asset, kopieren den Animations-Blueprint, legen die Rad- und Fahrzeug-Blueprints an, registrieren das Fahrzeug und hängen die Bordstein-Komponente an. Jedes Skript prüft sein Ergebnis selbst, etwa die Bounding Box nach dem Import oder die Radwerte einer neu erzeugten Instanz.

Zwei Eigenschaften von Unreal Engine 5.5 waren dabei zu berücksichtigen. Erstens liefert `BaseVehiclePawnNW` die Physiksimulation des Fahrzeug-Meshs ausgeschaltet; ohne die Einstellung `Simulate Physics` bleibt das Fahrzeug an der Spawnposition stehen. Zweitens übernehmen Instanzen einer Blueprint-Klasse nur die Werte, die beim letzten Kompilieren vom Elternobjekt abwichen. Die Rad-Blueprints

werden deshalb nach dem Setzen der Werte erneut kompiliert; andernfalls erhalten die Räder zur Laufzeit die Chaos-Standardwerte mit 32 cm Radius.

5.3 Fahrphysik und Antrieb

Die Fahrphysik wird durch Chaos Vehicles berechnet. Die Parametrierung umfasst Masse und Schwerpunkt, Rad- und Federungseigenschaften, den Antrieb an allen sechs Rädern sowie den Bordstein-Assistenten. Die gewählten Werte beschreiben einen Simulationsbetriebsfall und keine Herstellerkalibrierung.

5.3.1 Masse, Schwerpunkt und Fahrwerk

Die Simulation verwendet eine Fahrzeugmasse von 35 kg. Dieser Wert entspricht der veröffentlichten Gesamtmasse (Wevolver, n. d.); andere Quellen nennen für frühere Generationen geringere Leermassen. Am Fahrzeug-Mesh ist ein Schwerpunktversatz von $(0, 0, -15)$ cm eingestellt.

Tabelle 5.2: Wesentliche Geometrie- und Fahrwerksparameter des Starship-Modells unter Chaos Vehicles. Die Einstellungen stellen keine Herstellerkalibrierung dar.

Parameter	Wert
Fahrzeugmasse	35 kg
Radstand / Spurweite	0,382 / 0,512 m
Radradius / Radbreite	0,09 / 0,04 m
Maximaler Lenkwinkel vorne / Mitte / hinten	30° / 0° / 0°
Lenkgeometrie	Ackermann
Maximales Bremsmoment je Rad	20 Nm
Feststellbremsmoment je Mittel- und Hinterrad	40 Nm
Maximaler Einfederweg / Ausfederweg	18 / 4 cm
Federrate	14,3 N/cm
`Suspension Damping Ratio`	0,7
Rad-Boden-Abfrage	Spherecast

Die Vorderräder lenken; Mittel- und Hinterräder sind fest. Die Betriebsbremse wirkt auf alle sechs Räder, die Feststellbremse auf Mittel- und Hinterräder. Das Fahrzeug lenkt damit wie ein Pkw über die Vorderachse und nicht wie das Original über unterschiedliche Raddrehzahlen der Seiten.

Die Federung ist auf Bordsteine ausgelegt. Der große Einfederweg von 18 cm lässt ein Rad um eine 15 cm hohe Kante einfedern, ohne den harten Anschlag zu erreichen. Die Federrate folgt aus der Radlast und dem Ausfederweg, damit das Fahrzeug in Ruhe auf der modellierten Höhe steht:

$$k = \frac{m\,g/6}{s_{\text{aus}}} = \frac{35\,\text{kg} \cdot 9{,}81\,\text{m/s}^2/6}{4\,\text{cm}} \approx 14{,}3\,\text{N/cm}. \tag{5.1}$$

Chaos verarbeitet die Federrate intern in Zentimetern; der eingetragene Wert entspricht daher N/cm. Die Vorspannung wird im verwendeten Federungsmodus nicht ausgewertet.

Für die Rad-Boden-Kontakte werden Spherecast-Abfragen verwendet. Mit der zuvor genutzten Raycast-Abfrage tauchte das Rad an der Bordsteinfront ein und blieb hängen, da der Strahl nur einen Punkt unter der Radmitte erfasst. Die kugelförmige Abfrage erfasst die Kante bereits mit der Radkontur. Wie beim Clevon bestätigt dieser Befund die Funktion im betrachteten Testszenario, ersetzt aber keine vollständige fahrdynamische Validierung.

5.3.2 Allradantrieb

Alle sechs Räder werden angetrieben. Das Patent zum Überwinden von Hindernissen nennt mindestens vier Elektromotoren für die Räder (Liivik et al., 2024); der Antrieb aller sechs Räder ist eine Modellannahme, die die Traktion an Kanten verbessert. Das Differenzial ist auf `All Wheel Drive` mit einer Aufteilung von 50 % auf die Vorderachse eingestellt. Da Chaos Mittel- und Hinterräder derselben Achsgruppe zuordnet, erhalten die Vorderräder je 25 % und die übrigen vier Räder je 12,5 % des Antriebsmoments.

Der Ersatzmotor besitzt eine flache Drehmomentkennlinie mit `Max Torque` 3 Nm bis `Max RPM` 720 min^{-1}. Eine Vorwärtsgangübersetzung von 1 und eine Achsübersetzung von 4 ergeben mit dem Radradius $r = 0{,}09\,\mathrm{m}$ eine rechnerische Höchstgeschwindigkeit von

$$v_{\max} = \frac{n_{\max}}{i \cdot 60} \cdot 2\pi r = \frac{720}{4 \cdot 60}\,\mathrm{s}^{-1} \cdot 2\pi \cdot 0{,}09\,\mathrm{m} \approx 1{,}70\,\mathrm{m/s} \approx 6{,}1\,\mathrm{km/h}. \tag{5.2}$$

Diese Geschwindigkeit entspricht der Schrittgeschwindigkeit des Originals. Mit einem Getriebewirkungsgrad von 0,9 ergibt sich eine Zugkraft von $F = 3\,\mathrm{Nm} \cdot 4 \cdot 0{,}9 / 0{,}09\,\mathrm{m} = 120\,\mathrm{N}$. Bei 35 kg entspricht das rechnerisch bis zu $3{,}4\,\mathrm{m/s^2}$. Dieser Wert genügt für Anfahren und Steigungen, ohne dass die Räder ständig durchdrehen. Die Drehmomentkurve liegt als Kurven-Asset `Curve_Starship_Torque` vor; ohne Kurve schaltet Chaos die Antriebssimulation ab.

5.3.3 Bordstein-Assistent

Gehwege in der Referenzkarte Town10HD sind überwiegend durch 15 cm hohe Bordsteine von der Fahrbahn getrennt. Bei 9 cm Radradius und Schrittgeschwindigkeit überwindet das Chaos-Fahrwerk diese Kante nicht: Die Federkraft wirkt entlang der Flächennormale der fast senkrechten Bordsteinfront und damit überwiegend nach hinten. In drei Anfahrten ohne Unterstützung blieb das Fahrzeug jeweils vor der Kante stehen; das Abfahren gelang. Das Original löst das Problem mechanisch: Kipphebel verbinden Mittel- und Hinterrad, und ein eigener Motor dreht den Hebel und hebt dabei die Räder an (Liivik et al., 2024). Chaos kennt keine solchen Gelenke zwischen Rädern.

Die C++-Komponente `UStarshipCurbAssistComponent` im CARLA-Plugin ersetzt den Hebelmotor durch geregelte Hubkräfte an den Rädern. Sie wird in jedem Spieltakt vor der Physikberechnung ausgeführt und arbeitet in drei Schritten:

- **Erkennen:** Je Fahrzeugseite prüft eine waagrechte Strahlabfrage 5 cm über dem Boden den Bereich 10 bis 110 cm vor dem Vorderrad. Bei einem Treffer wird die Stufenhöhe bestimmt; Stufen von 4 bis 25 cm gelten als Bordstein. Ab der Erkennung ist die Annäherungsgeschwindigkeit auf 0,6 m/s begrenzt.

- **Anheben:** Ein PD-Regler hebt das jeweils nächste Rad auf eine Sollhöhe knapp über der Bordsteinoberkante. Die Kraft wirkt am einzelnen Rad und ist auf 150 N begrenzt. Zuerst wird die Vorderachse angehoben, danach Mittel- und Hinterachse. Die Sollhöhen liegen auf einer Geraden, sodass das bereits überquerte Rad aufsetzt und Vortrieb erzeugt.
- **Absichern:** Ab 40° Nickwinkel wird die Hubkraft vorne zurückgenommen, ab 6° Rollwinkel die höhere Seite entlastet.

Fahrzeuggeometrie, Masse und Höchstgeschwindigkeit bleiben unverändert. Die Komponente ist immer aktiv und wirkt damit auch beim Fahren mit dem CARLA-Fahrclient oder über ROS 2. Abbildung 5.4 zeigt den Ablauf.



(a) Anfahrt. **(b)** Vorderachse angehoben, 27°. **(c)** Auf dem Gehweg.

Abbildung 5.4: Auffahren auf einen 13,6 cm hohen Bordstein in Town10HD mit Bordstein-Assistent. Eigene Aufnahmen aus CARLA 0.10.0.

Der Assistent bildet die Wirkung der Hebel nach, nicht ihre Mechanik. Da der Aufbau starr bleibt, muss er beim Auffahren um mindestens etwa 28° kippen; das Original vermeidet das über die Gelenkhebel. Hindernisse bis etwa 25 cm Höhe werden als Bordstein behandelt.

5.4 Sensorik und Parametrierung

Das Sensorkonzept umfasst neun RGB-Kameras, drei ToF-Tiefensensoren, vier Radare, acht Ultraschallsensoren, GNSS und IMU. Die Anzahl von Kameras, Radaren und Ultraschallsensoren folgt veröffentlichten Angaben zum Starship (MIT Technology Review, 2018); ToF-Kameras werden in technischen Übersichten genannt (Wevolver, n. d.). Ein LiDAR ist nicht vorgesehen. Die virtuellen Sensoren werden als CARLA-Actors an das Fahrzeug gebunden. Ihre Positionen, Orientierungen und Attribute sind je Sensortyp in JSON-Profilen hinterlegt, die sowohl die Anzeigeprogramme als auch die ROS-Sensorskripte verwenden (Rauch & Le, 2026). Die ROS-2-Anbindung wird in Kapitel 7 beschrieben.

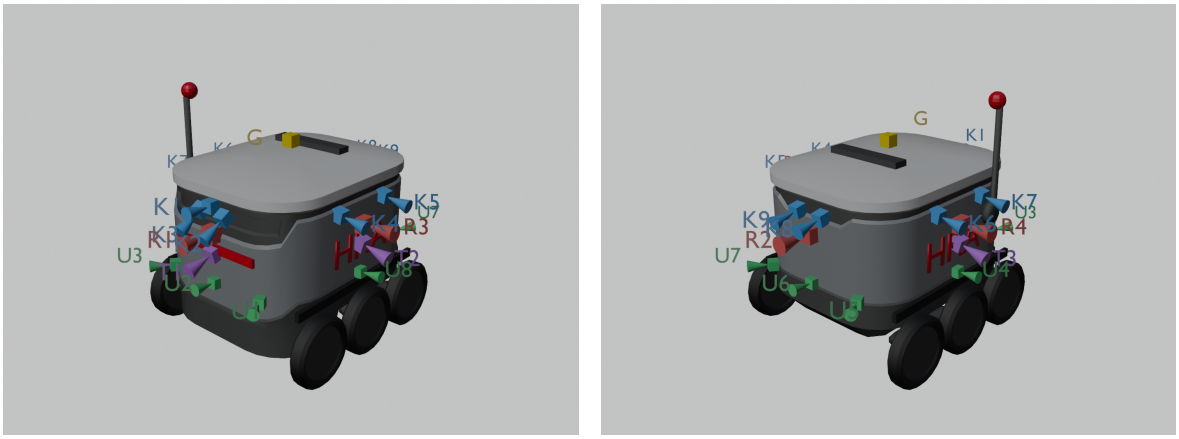
Tabelle 5.3 fasst die Profile zusammen. Die angegebenen Messraten sind Sollwerte in Simulationszeit; die tatsächlich erreichten Raten hängen vom Simulationstakt und der Rechenlast ab. Da keine Herstellerangaben zu den verbauten Sensoren vorliegen, sind Montagepunkte und Messparameter begründete Annahmen. Die Referenzprodukte dienen der Parametrierung und belegen nicht die Identität der verbauten Sensoren.

Die Anzeigeprogramme folgen dem Aufbau der Clevon-Skripte: Je Sensortyp zeigt ein eigenes Programm die Messdaten in umschaltbaren Reitern an. Die gemeinsamen Funktionen liegen im Modul `starship_sensor_common.py` mit derselben Schnittstelle wie beim Clevon. GNSS und IMU werden vom Fahrclient

Tabelle 5.3: Sensorprofile des Starship-Modells. Messraten sind Sollwerte in Simulationszeit.

Sensor	Wesentliche Parameter	Referenz
RGB, 9 Stück	640 × 480 Pixel (seitlich hochkant); 80° bzw. 60° horizontal; 4 Hz	Patent US20180253107A1
ToF, 3 Stück	Tiefenkamera 320 × 240 Pixel; 90°; 10 Hz; 5 m	Infineon/pmd REAL3
Radar, 4 Stück	20 m; 60° × 20°; 1500 Strahlen/s; 20 Hz	TI IWR6843AOP
Ultraschall, 8 Stück	Hindernissensor; 3 m; Radius 0,15 m; 20 Hz	MaxBotix MB1242
GNSS	5 Hz; Streuung ca. 2 m horizontal	Annahme
IMU	100 Hz; Rauschen MEMS-typisch	Annahme

`manual_control_starship.py` mitgeführt, der das CARLA-Beispielprogramm erweitert. Die Anzeigen für Kameras, ToF, Radar und Ultraschall sowie die GNSS- und IMU-Erweiterung wurden unter CARLA 0.10.0 funktional erprobt. Abbildung 5.5 zeigt die Anordnung.



(a) Ansicht von vorne links.

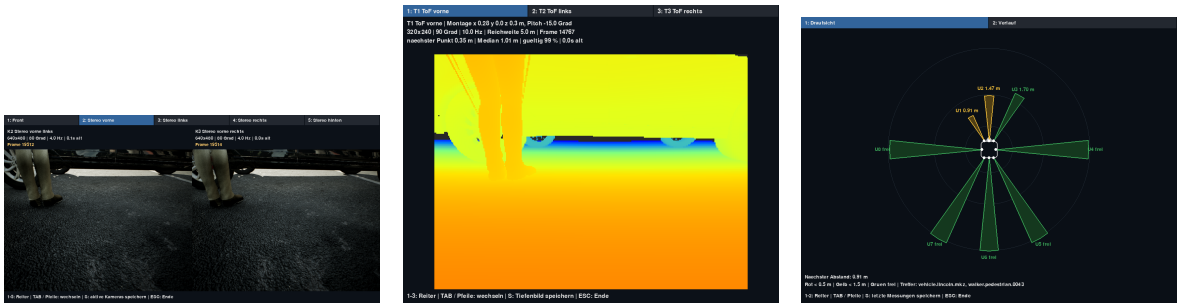
(b) Ansicht von hinten rechts.

Abbildung 5.5: Sensoranordnung am Starship-Modell: Kameras (K), ToF (T), Radar (R), Ultraschall (U) und GNSS (G); Kegel zeigen die Blickrichtung. Eigene Darstellung.

Abbildung 5.6 zeigt drei der Anzeigeprogramme während einer Testfahrt mit einem Fußgänger und einem Fahrzeug vor dem Bot.

5.4.1 Kameras und Blickrichtungen

Die Kameraanordnung folgt einem Patent des Herstellers (Heinla et al., 2018). Es beschreibt in der bevorzugten Ausführung neun Kameras: eine Einzelkamera vorne sowie je ein Stereopaar vorne, links, rechts und hinten. Die Stereobasis beträgt dort vorne und hinten 5 bis 10 cm, seitlich 15 bis 20 cm. Das vordere Paar ist 20 bis 40° nach unten geneigt, das Sichtfeld beträgt etwa 60° × 80° und die Bildrate 4 Bilder pro Sekunde. Seitliche Kameras sind hochkant angeordnet.



(a) Stereopaar vorne. **(b)** ToF vorne, nah = rot. **(c)** Ultraschall, Draufsicht.

Abbildung 5.6: Anzeigeprogramme der Starship-Sensorik in CARLA 0.10.0. Eigene Screenshots.

Im Modell beträgt die Stereobasis vorne und hinten 8 cm und seitlich 18 cm. Das vordere Paar ist um 30° nach unten geneigt, damit Bordstein und Boden direkt vor dem Fahrzeug im Bild liegen; die übrigen Paare um 10°. Die Einzelkamera blickt waagrecht nach vorne. Die Auflösung von 640 × 480 Pixeln und die Montagepunkte sind Annahmen, da das Patent dazu keine Angaben macht. Eine reale Linsenverzeichnung wird nicht nachgebildet.

5.4.2 ToF-Sensoren und Ultraschall

Für ToF und Ultraschall stellt CARLA keine eigenen Sensortypen bereit (Abschnitt 3.2). Die drei ToF-Sensoren sind als Tiefenkameras vorne und an beiden Seiten in 30 cm Höhe angebracht und um 15° nach unten geneigt. Auflösung und Sichtfeld orientieren sich an kompakten ToF-Bildsensoren wie der REAL3-Reihe von Infineon und pmd, etwa dem IRS2875C mit 240 × 180 Bildpunkten (Infineon Technologies AG, n. d.; TechInsights, n. d.); die Anzeige begrenzt die Messdistanz auf 5 m.

Die acht Ultraschallsensoren sind als Hinderniserkennungssensoren in 22 cm Höhe rundum verteilt: drei vorne, drei hinten und je einer an den Seiten. Die Reichweite orientiert sich an einem marktüblichen Ultraschallsensor (MaxBotix, n. d.) und ist auf 3 m begrenzt, da das Fahrzeug nur Schrittgeschwindigkeit fährt. Die Sensoren sind um 4° nach oben geneigt, da sie in dieser Höhe sonst die Fahrbahn als Hindernis melden. Der Sweep-Radius von 0,15 m bildet die Breite der Schallkeule nur grob nach.

5.4.3 Radar

Je ein Radar blickt nach vorne, hinten, links und rechts. Die Werte orientieren sich an einem 60-GHz-Kurzstreckenradar für Robotikanwendungen (Texas Instruments, n. d.). Das Sichtfeld ist mit 60° × 20° bewusst enger als beim Referenzprodukt gewählt, damit das CARLA-Radar weniger Bodenechos liefert. Die Reichweite beträgt 20 m. Je Sensor werden 1500 Strahlen pro Sekunde angesetzt; dieser Wert beschreibt die Abtastdichte der Simulation und keine Herstellerangabe. Wie beim Clevon liefert CARLA geometrische Treffer mit radialer Geschwindigkeit, keine vollständige Signalkette.

5.4.4 GNSS, IMU und Messraten

Ein GNSS-Empfänger wird in technischen Übersichten zum Starship genannt (Wevolver, n. d.). Eine IMU ist nicht belegt; sie wird wie beim Clevon als für die autonome Navigation notwendige Grundausstattung angenommen. Der GNSS-Sensor sitzt oben auf dem Deckel unter der Antenne, die IMU in der Fahrzeugmitte. Anders als beim Clevon ist Rauschen aktiviert: Die GNSS-Positionen streuen mit einer Standardabweichung von etwa 2 m horizontal und 1 m in der Höhe, entsprechend einem Empfänger ohne RTK-Korrektur. Die IMU erhält ein Rauschen, wie es für einfache MEMS-Sensoren typisch ist. Beide Werte sind Annahmen ohne Datenblatt. Die Sollraten betragen 5 Hz für GNSS und 100 Hz für die IMU. Tatsächliche Raten hängen von den Simulationsschritten ab; insbesondere sind 100 Hz IMU-Ausgabe im asynchronen Betrieb nicht zugesichert (Abschnitt 7.7).

5.5 Machbarkeit weiterer Funktionen

Zwei Funktionen des Originals sind im Modell nicht umgesetzt, aber mit den vorhandenen Werkzeugen grundsätzlich realisierbar. Beide wurden nicht erprobt.

Gehwegnavigation. CARLA bewegt Fußgänger über ein Navigationsnetz der Karte, gesteuert durch einen eigenen KI-Controller, der nur für Fußgänger-Actors vorgesehen ist. Fahrzeuge folgen dagegen über den Autopiloten den Fahrspuren der Straße. Die Trajektorien der Fußgänger lassen sich daher nicht direkt auf den Bot übertragen; mit Autopilot fährt er wie ein Pkw auf der Fahrbahn. Machbar ist eine eigene Bahnführung: Die Karte enthält Gehwege als eigenen Spurtyp, aus dem sich Wegpunkte abfragen lassen. Ein Python- oder ROS-Knoten kann diesen Wegpunkten mit einem einfachen Bahnfolgeregler folgen und Gas und Lenkung über die vorhandene Schnittstelle senden. Alternativ kann ein unsichtbarer, KI-gesteuerter Fußgänger die Route vorgeben, der der Bot folgt. Zu berücksichtigen sind der Wenderadius von 2,2 bis 2,4 m, der enge Kurven ausschließt, sowie Bordsteine und Straßenquerungen, die der Bordstein-Assistent bzw. eine eigene Querungslogik abdecken müssen.

Ansteuerung einzelner Räder und Achsen. Die CARLA-Fahrzeugsteuerung überträgt nur gemeinsame Werte für Gas, Lenkung und Bremse. Chaos Vehicles bietet jedoch Funktionen, um Antriebs- und Bremsmoment je Rad zu setzen. Eine C++-Komponente nach dem Vorbild des Bordstein-Assistenten könnte den Lenkbefehl in unterschiedliche Momente der linken und rechten Seite übersetzen und so eine Panzerlenkung mit Drehen auf der Stelle nachbilden; der Lenkwinkel der Vorderräder wäre dann null. Auf dieselbe Weise ließen sich Vorder-, Mittel- und Hinterachse getrennt antreiben. Offen ist, ob das Reifenmodell von Chaos den dafür nötigen seitlichen Schlupf realistisch abbildet; gegebenenfalls ist die Seitenführung der Mittelräder zu verringern. Für eine Ansteuerung über ROS 2 wäre eine eigene Nachricht mit Werten je Seite oder Rad erforderlich.

6 Implementierung des Clevon 1

Dieses Kapitel beschreibt die Modellierung und Integration des Clevon 1 in CARLA. Im Mittelpunkt stehen die Fahrzeuggeometrie, die Umsetzung der Fahrphysik mit Chaos Vehicles sowie Sensorik, Beleuchtung und Ladeklappe. Beschrieben wird der abschließende Implementierungsstand unter CARLA 0.10.0 und Unreal Engine 5.5.4; die ROS-2-Anbindung folgt in Kapitel 7.

6.1 Referenzfahrzeug und Modellumfang

Der Clevon 1 bildet den straßenbasierten Lieferroboter des Projekts ab. Das Herstellerproduktblatt nennt einen elektrischen Hinterradantrieb mit 2×3 kW und eine Geschwindigkeit von 50 km/h (Clevon, n. d. b, S. 2). Der dargestellte CargoBox-Aufbau besitzt laut eigenem Datenblatt ein Ladevolumen von 1 m^3 und eine Nutzlast bis 150 kg (Clevon, n. d. a). Diese Herstellerangaben dienen als Referenz für die Modellierung und Parametrierung. Das Simulationsmodell umfasst die Fahrzeuggeometrie mit CargoBox, die Fahr- und Lenkfunktionen, Beleuchtung, eine bewegliche Ladeklappe sowie virtuelle Sensoren.

6.1.1 Eigene Modellierung und Herkunft der Bauteile

Die Fahrzeugkarosserie und der CargoBox-Aufbau wurden in Blender eigenständig anhand von Referenzbildern nachmodelliert (Abbildung 6.1). Sichtbare Konturen und Proportionen wurden möglichst weitgehend nachgebildet und an den verfügbaren Herstellermaßen ausgerichtet. Die Geometrie ist damit eine bildgestützte Rekonstruktion und wurde nicht als vollständiges Clevon-CAD-Modell übernommen. Die Tutorials von Gryd CG (n. d.) und Gallant (n. d.) dienten als methodische Hilfen zur Fahrzeugmodellierung in Blender, nicht als Quellen für die Fahrzeugabmessungen.

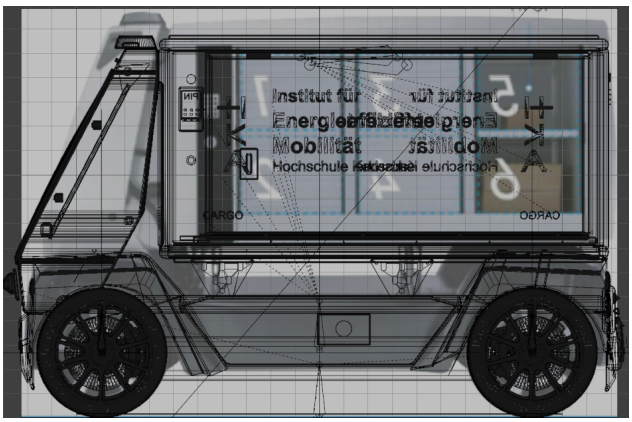


Abbildung 6.1: Bildgestützte Rekonstruktion der Clevon-Karosserie (hier noch ohne LiDAR). Referenzbild: Clevon (2023, S. 7); Modellansicht: eigene Darstellung.

Der Felgensatz stammt aus dem Modellpaket *Car wheel pack* von renderZone (renderZone, 2024). Er wird als übernommenes Bauteil von der eigenen Karosserie-

modellierung unterschieden. Er wurde durch Skalieren in das Modell übernommen, bildet jedoch nicht den originalen Felgensatz des Clevon ab.

Karosserie, Räder, Bremssättel, Glas, Leuchten und Ladeklappe wurden für ihre unterschiedlichen Funktionen getrennt aufgebaut. Abbildung 6.2 zeigt das Gesamtmodell mit CargoBox und ergänzendem Dach-LiDAR als gerendertes Blender-Objekt. Die modellierten Sensorgehäuse dienen der visuellen Darstellung; ihre Messfunktion wird erst durch die zugeordneten virtuellen Sensoren in CARLA bereitgestellt.



Abbildung 6.2: Clevon-Modell mit CargoBox und ergänzendem Dach-LiDAR. Eigene Darstellung aus Blender; neutrale Materialansicht.

6.2 Geometrie, Rig und CARLA-Integration

Das Fahrzeugskelett besteht aus dem Basisknochen `Vehicle_Base` und vier Radknochen, die jeweils im Radmittelpunkt liegen. Die zugehörigen Geometrien sind starr an die Knochen gebunden. Im Chaos-Animations-Blueprint `AnimBP_Clevon1_Chaos` überträgt der *Wheel Controller* die berechneten Lenk-, Roll- und Federbewegungen auf die Radknochen. Vereinfachte Kollisionskörper repräsentieren die Karosserie. Die Rad-Boden-Kontakte werden getrennt davon durch die Chaos-Fahrzeugbewegungskomponente ermittelt. Hierfür wird *Spherecast* verwendet, nachdem diese Abfrage im Funktionstest das beobachtete Eintauchen der Reifen in Bordsteine behoben hatte. Ein zusätzlicher Kollisionsproxy dient Sensor- und Raycast-Abfragen. Sichtbare Geometrie, Karosseriekollision und Rad-Boden-Abfrage erfüllen damit unterschiedliche Aufgaben.

Die Anordnung des Basisknochens und der vier Radknochen zeigt Abbildung 6.3.

Die Modellquellen und Exportdateien liegen im Repository unter `01_Clevon1/Modell`. Die Blender-Datei `Clevon1_Funktionen_Rueckfahrlicht.blend` enthält den korrigierten Modellstand mit weißer Rückfahrlichtfläche. Die zugehörigen Glas- und Leuchtenexporte liegen als `SM_Clevon1_Glass.fbx` und `SM_Clevon1_Lights.fbx` neben den weiteren bauteilweise getrennten Exportdateien im Unterordner `Modell/FBX`.

Der FBX-Export verwendet Zentimeter; der Importfaktor beträgt 1. Die Skalierung von Mesh und Skelett beträgt jeweils 1/1/1. Glas, Leuchten, Ladeklappe und Bremssättel liegen in Unreal als separate Komponenten vor. Bei der Migration konnten Geometrie und Skelett aus dem bestehenden UE4-Modell übernommen werden; die

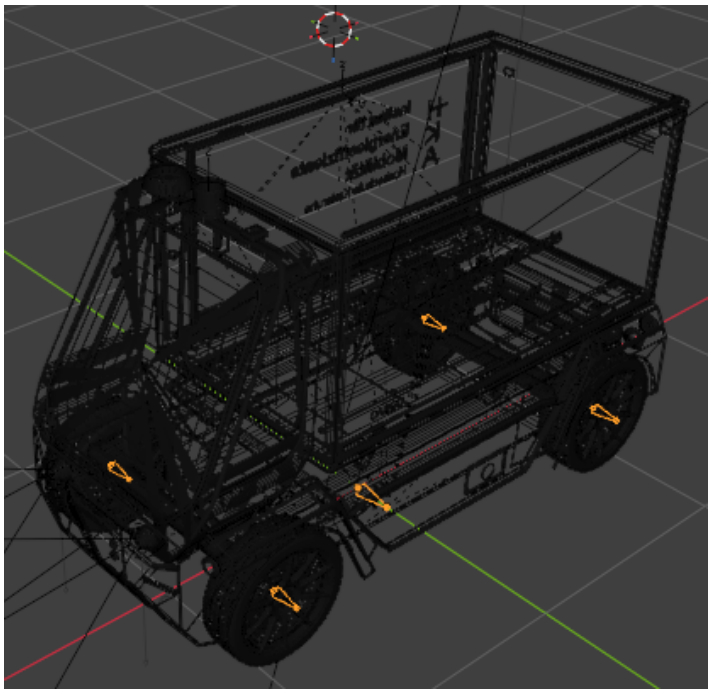


Abbildung 6.3: Anordnung des Basisknochens `Vehicle_Base` und der vier Radknochen im Clevon-Modell. Eigene Darstellung aus Blender.

Radklassen und die Radanimation wurden für Chaos neu eingerichtet. Die Bauteile wurden aus der Blender-Datei in separate FBX-Dateien exportiert, damit sie in Unreal Engine unabhängig importiert, mit Materialien versehen und entsprechend ihrer Funktion animiert oder angesteuert werden können.

Der Fahrzeug-Blueprint `BP_Clevon1` basiert auf `BaseVehiclePawn`. Seine Registrierung erfolgt über die `VehicleParameters.json` unter der Kennung `vehicle.clevon.1`. Materialien, Lichtsteuerung und bewegliche Zusatzteile werden in Unreal Engine eingerichtet und sind nicht allein durch den FBX-Import funktionsfähig.

6.2.1 Abmessungen

Tabelle 6.1 vergleicht die Herstellerangaben mit den Außenmaßen der ausgewerteten Blender-Geometrie im dokumentierten Exportstand. Diese wurden anhand einer gemeinsamen Bounding-Box der berücksichtigten Bauteile bei geschlossener Ladeklappe und in Ruhepose bestimmt. Die Sensoranordnung ist in Anhang A dargestellt.

Tabelle 6.1: Abmessungen gemäß Herstellerproduktblatt (Clevon, n. d. b, S. 2) und eigener Geometriemessung.

Größe	Referenz	Modell	Abweichung
Länge	2,470 m	2,570 m	+4,1 %
Breite	1,150 m	1,168 m	+1,6 %
Höhe ohne LiDAR	1,550 m	1,571 m	+1,4 %
Höhe mit LiDAR	–	1,618 m	–

Die Abweichungen kennzeichnen die begrenzte Maßtreue der bildgestützten Rekonstruktion. Perspektivische Verkürzungen, verdeckte Flächen und nicht eindeutig erkennbare Konturen können die Modellierung beeinflussen. Die verfügbaren Gesamtmaße legen zudem nicht sämtliche Einzelformen fest. Die von CARLA gemeldete Actor-Bounding-Box beträgt etwa $2{,}555 \times 1{,}168 \times 1{,}618$ m.

6.2.2 Aufgaben der Blueprint-Bausteine

Das Fahrzeug entsteht in Unreal Engine aus dem Zusammenspiel mehrerer Assets. Das Skeletal Mesh enthält die sichtbare Grundgeometrie und das Fahrzeugskelett. Das Physics Asset beschreibt die zugehörigen physikalischen Körper. Vier Chaos-Wheel-Blueprints legen die radbezogenen Parameter fest, darunter Radius, Lenkfähigkeit, Bremsmomente und Federung. Die Chaos-Fahrzeugbewegungskomponente berechnet die Fahrzeugbewegung; der Animations-Blueprint `AnimBP_Clevon1_Chaos` überträgt die daraus abgeleiteten Radbewegungen auf die Radknochen. Der Fahrzeug-Blueprint `BP_Clevon1` verbindet diese Bausteine und ergänzt die fahrzeugspezifischen Funktionen.

Die Komponentenstruktur umfasst neben dem Fahrzeug-Mesh separate Glas- und Leuchtenmeshes, vier Bremssättel, die Ladeklappe mit ihrem Constraint sowie zusätzliche Lichtquellen (Abbildung 6.4). Diese Aufteilung ermöglicht die getrennte Ansteuerung der beweglichen Bauteile und Lichtfunktionen. Erst die Zuordnung der Assets und das Zusammenspiel von Fahrzeugbewegung, Animation und Blueprint-Logik ergeben das funktionsfähige Fahrzeugmodell.

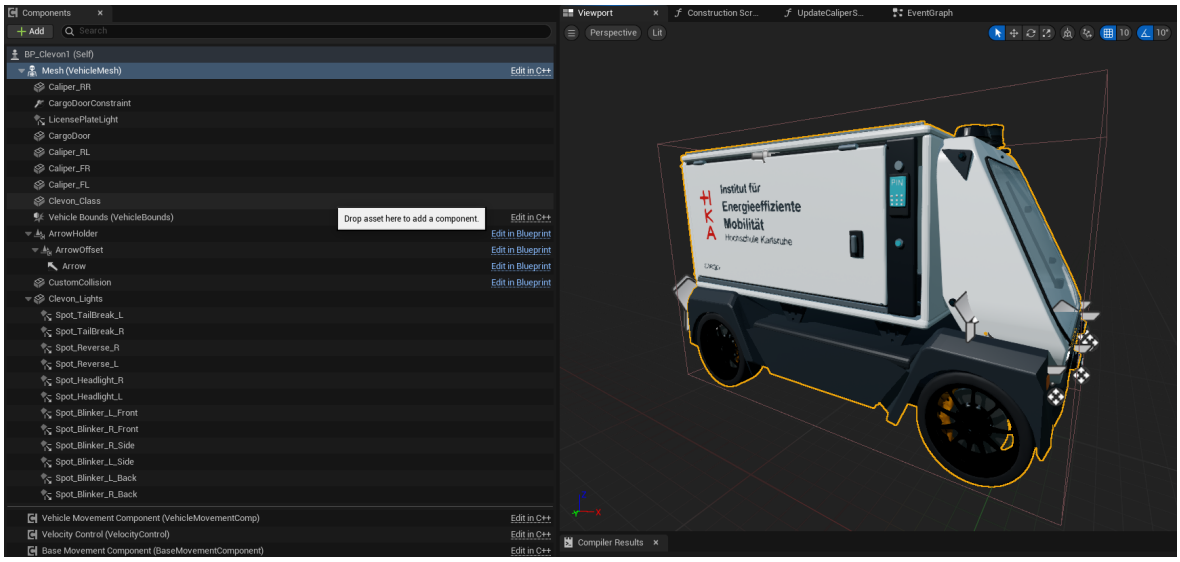


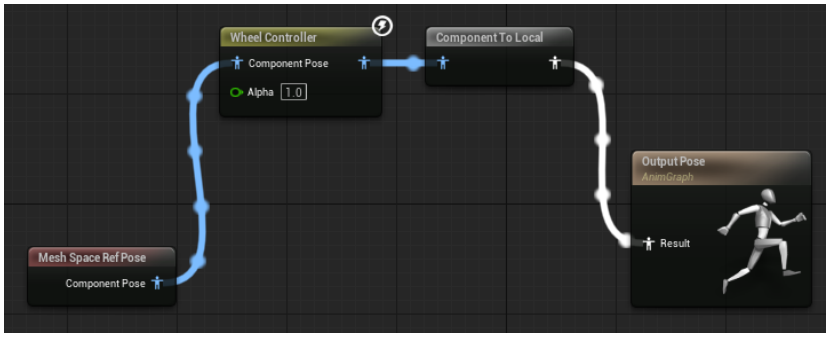
Abbildung 6.4: Komponentenstruktur des Clevon-Fahrzeug-Blueprints in Unreal Engine 5.5.4. Eigener Screenshot.

6.2.3 Skelettanimation und Kollisionskörper

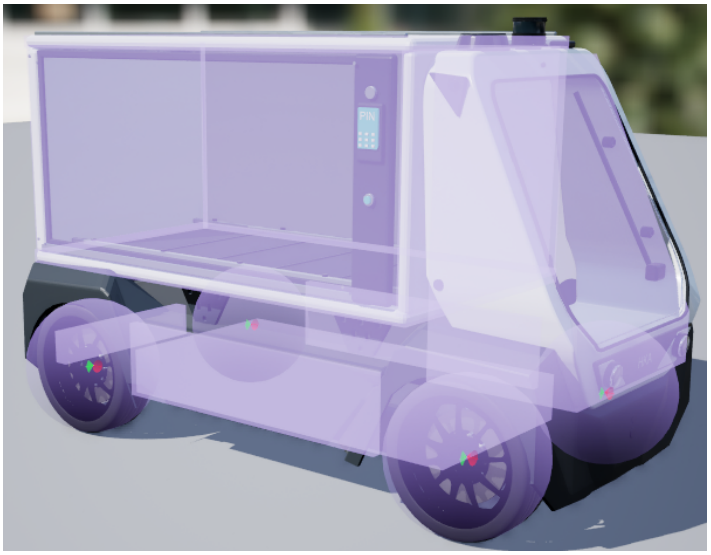
Der Animations-Blueprint `AnimBP_Clevon1_Chaos` verwendet den Chaos-Knoten *Wheel Controller*. Der Animationsgraph verbindet *Mesh Space Ref Pose*, *Wheel Controller*, *Component To Local* und *Output Pose*. Damit werden Lenkung, Raddrehung

und Federbewegung aus dem simulierten Fahrzeugzustand auf die Radknochen übertragen. Eine vorgefertigte Bewegungsanimation ist dafür nicht erforderlich.

Das Physics Asset enthält vereinfachte Kollisionskörper für die Karosserie und die vier Räder. Die Radkörper sind kinematisch eingestellt; ihre Kollisionsantwort ist deaktiviert. Die Rad-Boden-Kontakte werden stattdessen durch die Spherecast-Abfragen der Chaos-Fahrzeugbewegungskomponente ermittelt. Die Karosseriekörper bilden die übrigen mechanischen Kontakte ab. Sichtbare Details benötigen deshalb nicht jeweils einen eigenen physikalischen Körper.



(a) Chaos-Animationsgraph zur Übertragung der simulierten Radbewegungen auf das Fahrzeugskelett.



(b) Physics Asset mit vereinfachten Karosserie- und Radkörpern. Die Kollisionsantwort der Radkörper ist deaktiviert.

Abbildung 6.5: Radanimation und Kollisionskörper des Clevon in Unreal Engine 5.5.4. Die dargestellten Radkörper sind von den Spherecast-Abfragen für den Rad-Boden-Kontakt zu unterscheiden. Eigene Screenshots.

6.2.4 Event Graph und Aktualisierungsablauf

Die Blueprint-Logik trennt einmalige Initialisierung, laufende Bewegungsaktualisierung und die Verarbeitung von Zustandsänderungen. Das Construction Script ergänzt diese Abläufe um die Positionierung der Bremssättel in der Editoransicht.

- Das *Construction Script* aktualisiert die Bremssattelpositionen bereits vor dem Simulationsstart.

- `BeginPlay` erzeugt beim Start dynamische Materialinstanzen für die Leuchtflächen. Die gespeicherten Referenzen ermöglichen die spätere Anpassung ihrer Emission.
- `Event Tick` ruft nach dem Parent-Tick die Funktionen zur Aktualisierung der Bremssattelpositionen und anschließend der vorderen Bremssattellenkung auf.
- `Refresh Light State` verarbeitet Änderungen der CARLA-Lichtzustände. Ein wiederholender Timer aktualisiert zusätzlich die Blinkphase.

Damit werden Lichtzustände ereignisabhängig ausgewertet, während die Bremssättel den laufend veränderten Radstellungen folgen. Die geerbten Fahrzeugfunktionen bleiben im Ablauf eingebunden. Diese Ablaufstruktur konnte bei der Migration auf Unreal Engine 5 weitgehend übernommen werden. Die Anpassung an Chaos betrifft insbesondere den Zugriff auf die simulierten Radwinkel innerhalb der Bremssattellenkung. Die Trennung zwischen laufender Bewegungsaktualisierung und zustandsabhängiger Lichtsteuerung bleibt erhalten. Der Aufruf des Parent-Ticks bindet weiterhin die geerbte Tick-Verarbeitung ein.

Abbildung 6.6 soll die relevanten Einstiegsereignisse und Funktionsaufrufe zeigen; ein vollständig herausgezoomter Graph wäre für die Beschreibung der Logik kaum lesbar. Die nachfolgenden Abschnitte erläutern die Bremssattel- und Lichtfunktionen im Detail.

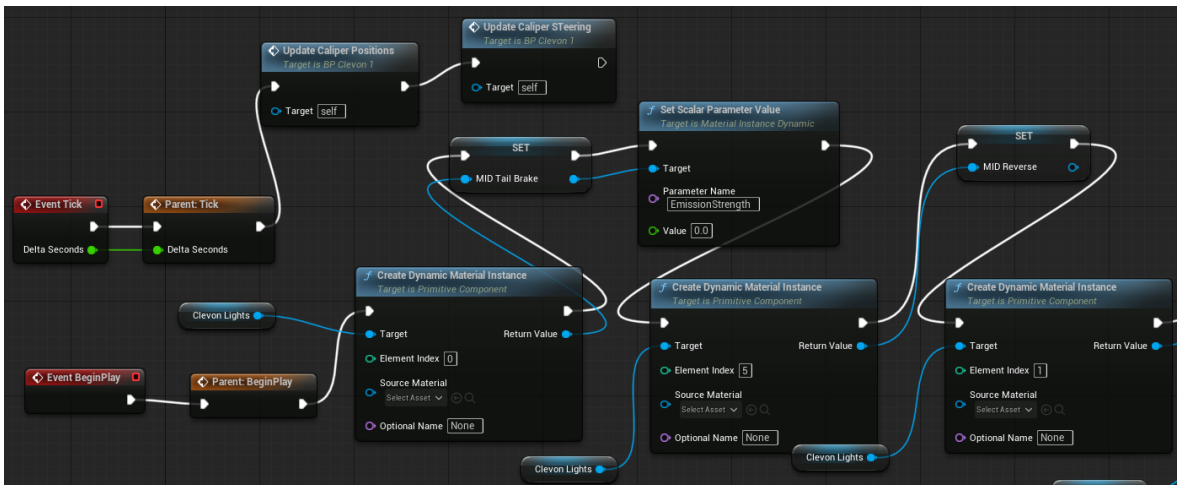


Abbildung 6.6: Ereignisse und Funktionsaufrufe im Event Graph von `BP_Clevon1` in Unreal Engine 5.5.4. Eigene Screenshots.

6.2.5 Bremssättel: Federbewegung und Lenkung

Die Bremssättel sollen der Federbewegung und vorne zusätzlich dem Lenkeinschlag folgen, jedoch nicht mit der Felge um die Radachse rotieren. Sie sind deshalb als separate Mesh-Komponenten am Fahrzeug-Mesh angebracht. Position und Orientierung werden getrennt aktualisiert, da eine vollständige Übernahme der Radknochentransformation auch die unerwünschte Rollrotation übertragen würde.

`UpdateCaliperPositions` liest die Transformation jedes Radknochens mit `Get Socket Transform` im *Component Space* des Fahrzeug-Meshs aus. Über `Break Transform` wird ausschließlich die Position entnommen und mit `Set Relative`

`Location` auf den zugehörigen Bremssattel übertragen. Da die Bremssättel dem Fahrzeug-Mesh untergeordnet sind, beziehen sich beide Angaben auf denselben Komponentenraum. So folgen die Bremssättel der Radposition beim Ein- und Ausfedern. Diese Positionsübertragung konnte bei der Migration beibehalten werden.

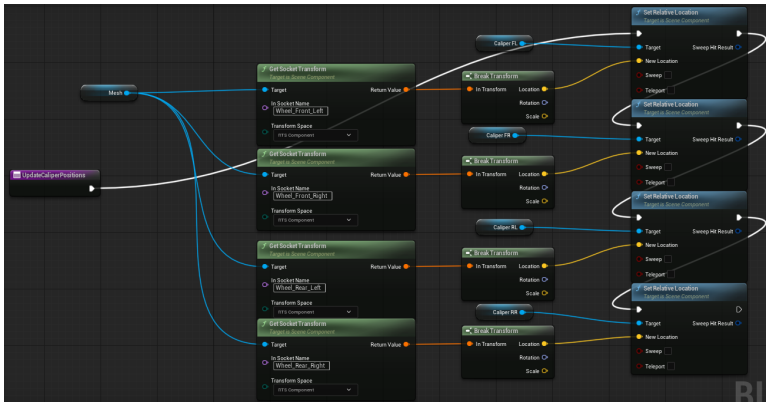
Für die Lenkbewegung wurde der Zugriff auf das Fahrzeugbewegungsmodell an Chaos angepasst. Die Funktion zur Bremssattellenkung ermittelt mit `Get Component by Class` die `ChaosWheeledVehicleMovementComponent` und liest deren `Wheels`-Array aus. Die Indizes 0 und 1 entsprechen dem linken und rechten Vorderrad. Vor dem Zugriff prüft `Is Valid Index`, ob der zweite Array-Eintrag vorhanden ist.

Der Knoten `Get Steer Angle` liefert den Lenkwinkel der jeweiligen `ChaosVehicleWheel`-Instanz. Dieser wird mit `Make Rotator` als Yaw-Rotation übernommen; Roll und Pitch bleiben null. `Set Relative Rotation` überträgt die Orientierung auf den jeweiligen vorderen Bremssattel. Die hinteren Bremssättel erhalten keinen Lenkeinschlag.

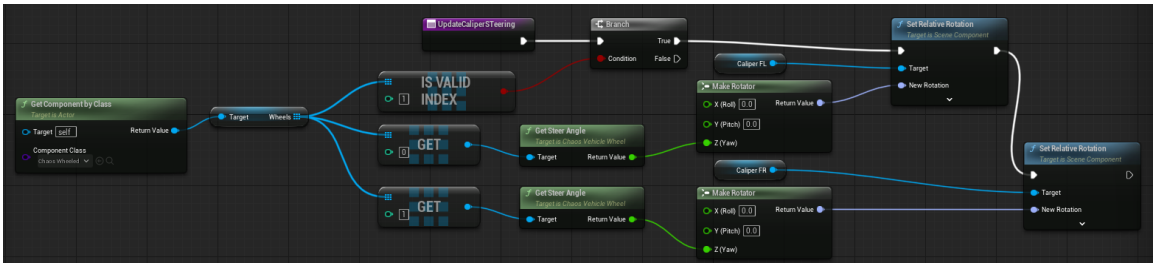
Damit bleiben Federbewegung, Lenkung und Rollrotation getrennt. Die Blueprint-Logik steuert ausschließlich die sichtbare Bewegung der Bremssättel. Das physikalische Bremsmoment wird unabhängig davon durch die Bremsparameter der Chaos-Radklassen bestimmt.



(a) Vorderrad mit separat modelliertem Bremssattel.



(b) Übertragung der Radknochenposition auf den linken vorderen Bremssattel.



(c) Abfrage der Chaos-Radlenkwinkel und Übertragung auf die vorderen Bremssättel.

Abbildung 6.7: Getrennte Aktualisierung von Position und Lenkung der Bremssättel in Unreal Engine 5.5.4. Die Rollrotation der Radknochen wird nicht übernommen. Eigene Screenshots.

6.3 Fahrphysik und Antrieb

Die Fahrphysik wird durch Chaos Vehicles berechnet. Die Parametrierung umfasst Masse und Schwerpunkt, Rad- und Federungseigenschaften sowie das aggregierte Antriebsmodell. Die gewählten Werte beschreiben einen Simulationsbetriebsfall und keine vollständige Herstellerkalibrierung.

6.3.1 Masse, Schwerpunkt und Fahrwerk

Die Simulation verwendet eine Fahrzeugmasse von 450 kg als angesetzten Betriebsfall. Dieser Wert ist keine belegte Leermasse: Das Produktblatt nennt etwa 300 kg netto und 470 kg gesamt, während die DHL-Fallstudie 320 kg netto und 530 kg gesamt aufführt (Clevon, n. d. b, 2023). 450 kg schienen in der Betrachtung als Fall einer durchschnittlichen und erwartbaren Masse durch Zuladung

Am Fahrzeug-Mesh ist ein Schwerpunktversatz von $(0, 0, -50)$ cm eingestellt. Dieser verschiebt den berechneten Körperschwerpunkt nach unten und bezeichnet keine absolute Schwerpunktposition. Der zusätzliche Schwerpunkt-Override der Fahrzeugbewegungskomponente ist deaktiviert.

Tabelle 6.2: Wesentliche Geometrie- und Fahrwerksparameter des Clevon-Modells unter Chaos Vehicles. Die Einstellungen stellen keine vollständige OEM-Kalibrierung dar.

Parameter	Wert
Fahrzeugmasse	450 kg
Radstand / Spurweite	1,770 / 0,966 m
Radradius	0,267 m
Radbreite	0,165 m
Maximaler Lenkwinkel vorne / hinten	50° / 0°
Lenkgeometrie	Ackermann
`Friction Force Multiplier`	2,0
Maximales Bremsmoment je Rad	450 Nm
Feststellbremsmoment je Hinterrad	900 Nm
Maximaler Einfederweg / Ausfederweg	8 / 8 cm
`Suspension Damping Ratio`	0,8
Rad-Boden-Abfrage	Spherecast
Abgefragte Kollisionsgeometrie	SimpleSweep

Die Vorderachse übernimmt die Lenkung, während die Hinterräder angetrieben werden. Die Lenkgeometrie ist auf Ackermann eingestellt. Eine geschwindigkeitsabhängige Lenkkennlinie begrenzt den verfügbaren Lenkeinschlag bei zunehmender Geschwindigkeit. Die Betriebsbremse wirkt auf alle vier Räder, die Feststellbremse ausschließlich auf die Hinterräder.

Die Federung erlaubt je Rad einen Ein- und Ausfederweg von jeweils 8 cm. Das dimensionslose Dämpfungsverhältnis beträgt 0,8. Federsteifigkeit, Vorspannung und Reifenkräfte sind bislang nicht anhand von Messungen am realen Fahrzeug kalibriert. Der `Friction Force Multiplier` ist ein Skalierungsparameter des Chaos-Reifenmodells und kein unmittelbar gemessener Reifen-Fahrbahn-Reibwert.

Für die Rad-Boden-Kontakte werden Spherecast-Abfragen gegen die einfache Kollisionsgeometrie der Umgebung verwendet. Die kugelförmige Abfrage berück-

sichtigt den Radradius, bildet jedoch nicht die vollständige Reifengeometrie ab. Im praktischen Fahrtest wurde damit das beobachtete Eintauchen der Reifen in Bordsteine behoben. Dieser Befund bestätigt die Funktion im betrachteten Testszenario, ersetzt aber keine vollständige fahrdynamische Validierung.

Die sichtbare Bewegung der Bremssättel ist von den physikalischen Brems- und Federungsparametern getrennt. Sie folgt der in Abschnitt 6.2.5 beschriebenen Blueprint-Logik.

6.3.2 Aggregiertes 6-kW-Modell

Die beiden im Herstellerproduktblatt angegebenen Antriebe mit jeweils 3 kW werden durch einen gemeinsamen Ersatzmotor mit einer achtpunktigen Drehmomentkennlinie angenähert (Clevon, n. d. b). Bei niedriger Drehzahl beträgt das Drehmoment rund 58,2 Nm; mit zunehmender Drehzahl sinkt es auf etwa 11,1 Nm bei 5000 min^{-1}. Die Stützstellen sind so skaliert, dass auch bei linearer Interpolation zwischen ihnen die mechanische Kennlinienleistung näherungsweise 6 kW nicht überschreitet:

$$P(n) = M(n)\,\frac{2\pi n}{60}. \tag{6.1}$$

Dabei ist n in min^{-1} und $M(n)$ in Nm einzusetzen; $P(n)$ ergibt sich in Watt. Die Kennlinie beschreibt eine mechanische Modellleistung, keine elektrische Leistungsaufnahme oder gemessene Originalkennlinie. Das Herstellerblatt unterscheidet nicht zwischen Dauer- und Spitzenleistung.

In Chaos wird die Kennlinie durch das Kurven-Asset `Curve_Clevon1_Torque` und den Parameter `Max Torque` abgebildet. Dieser beträgt 58,2164 Nm. Die Drehmomentwerte der Kurve sind durch diesen Maximalwert geteilt und damit dimensionslos:

$$m_{\mathrm{norm}}(n) = \frac{M(n)}{M_{\mathrm{max}}}. \tag{6.2}$$

Die horizontale Achse enthält weiterhin die Drehzahl in min^{-1}. Die Stützstellen sind linear interpoliert; `Max RPM` ist auf 5000 eingestellt.

Der Ersatzmotor wirkt über ein Hinterraddifferenzial auf die beiden Hinterräder. Eine Vorwärtsgangübersetzung von 1 und eine Achsübersetzung von 10 ergeben bei 5000 min^{-1} und einem Radradius von 0,267 m eine rechnerische Geschwindigkeit von 50,3 km/h ohne Schlupf. Diese kinematische Beziehung stellt weder einen harten Geschwindigkeitsbegrenzer noch einen Nachweis der tatsächlich erreichten Höchstgeschwindigkeit dar.

Der Getriebewirkungsgrad ist als Modellannahme auf 0,9 eingestellt.

6.4 Sensorik und Parametrierung

Das Sensorkonzept umfasst sechs RGB-Kameras, zwei Front-Radare, GNSS und IMU. Kameras und Radare folgen der Darstellung der Sichtbereiche im Herstellerproduktblatt (Clevon, n. d. b, S. 2); die Anzahl von sechs Kameras ist aus den dort gezeigten Front-, Seiten- und Rundumsichtbereichen abgeleitet. Der LiDAR ist eine ergänzende Simulationsvariante. Ultraschall und eine Simulation der 4G-Verbindung

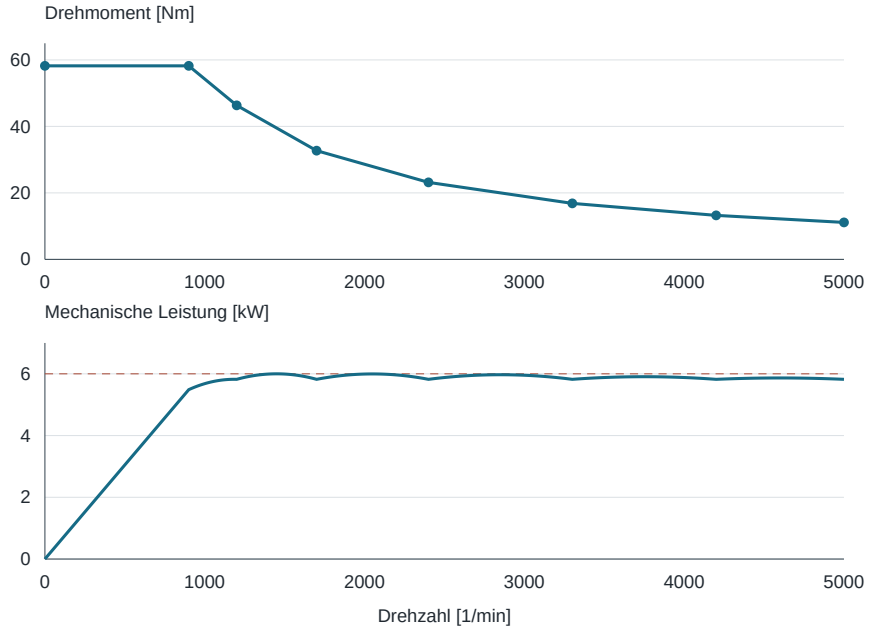


Abbildung 6.8: Drehmoment und daraus berechnete mechanische Leistung des 6-kW-Ersatzmodells. Eigene Berechnung aus den gespeicherten Stützstellen.

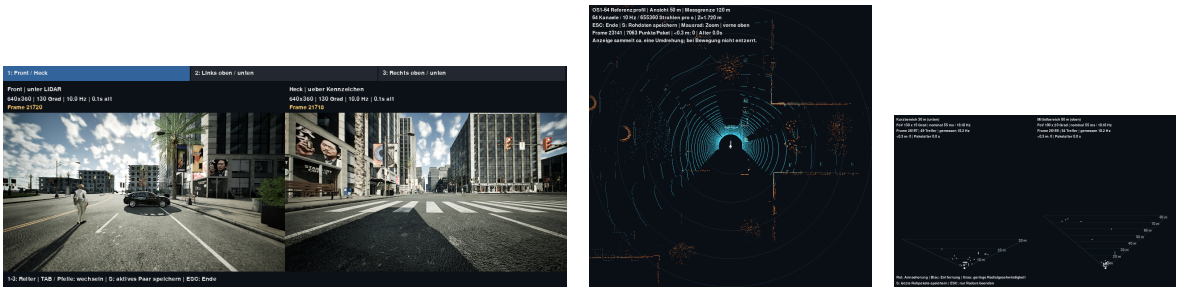
sind nicht enthalten. Die virtuellen Sensoren werden als CARLA-Actors an das Fahrzeug gebunden. Ihre Positionen, Orientierungen und Attribute sind in JSON-Profilen hinterlegt, die sowohl die Anzeigeprogramme als auch die ROS-Sensorskripte verwenden (Rauch & Le, 2026). Die ROS-2-Anbindung wird in Kapitel 7 beschrieben.

Tabelle 6.3 fasst die Referenzprofile zusammen. Die angegebenen Messraten sind Sollwerte in Simulationszeit; die tatsächlich erreichten Raten hängen vom Simulationstakt und der Rechenlast ab. Die Referenzprodukte dienen der Parametrierung und belegen nicht die Identität der am realen Clevon verbauten Sensoren.

Tabelle 6.3: Sensorprofile des Clevon-Modells. Messraten sind Sollwerte in Simulationszeit; beim LiDAR ist die Rotationsfrequenz angegeben.

Sensor	Wesentliche Parameter	Referenz
RGB, 6 Stück	1920 × 1080 Pixel; 130° horizontal; 30 Hz	e-con Systems und Leopard Imaging
Radar kurz	30 m; 130° × 15°; 55 ms	Clevon / DRVEGRD 169
Radar mittel	80 m; 100° × 20°; 55 ms	Clevon / DRVEGRD 152
LiDAR	64 Kanäle; 120 m; 10 Hz Rotation; ±22,5° vertikal	Ouster OS1 Rev. 05
GNSS	5 Hz	u-blox ZED-F9P
IMU	100 Hz	Bosch SMI230

Zur Verringerung der Renderlast erzeugt die Kameraanzeige jeweils nur die beiden Kameras des ausgewählten Reiters. Die sechs Kamerapositionen bleiben im Profil hinterlegt. Die Anzeigeprogramme für Kameras, Radar und LiDAR sowie die GNSS- und IMU-Erweiterung des Fahrclients wurden unter CARLA 0.10.0 funktional erprobt (Abbildung 6.9).



(a) Kamera-Reiter Front und Heck. **(b)** LiDAR, Draufsicht. **(c)** Front-Radare.

Abbildung 6.9: Anzeigeprogramme der Clevon-Sensorik in CARLA 0.10.0. Eigene Screenshots.

6.4.1 Kameras und Blickrichtungen

Eine Kamera sitzt vorne unterhalb des ergänzenden LiDARs, eine weitere hinten oberhalb des Kennzeichens. Auf jeder Kabinenseite befinden sich zwei Kameras. Die oberen blicken seitlich, die unteren sind gegenüber der seitlichen Blickrichtung um 45° nach hinten gedreht. Alle vier Seitenkameras sind um 10° nach unten geneigt. Diese Anordnung ist eine geometrische Annäherung und keine vermessene OEM-Kalibrierung. Positionen und Blickrichtungen zeigt die Zeichnung auf S. 45.

Die Auflösung von 1920 × 1080 Pixeln und die Sollrate von 30 Bildern pro Simulationssekunde orientieren sich am AR0230-Modul von e-con Systems. Das horizontale Sichtfeld von 130° folgt dem Referenzmodul LI-AR0233-GMSL3-130H von Leopard Imaging (e-con Systems, n. d.; Leopard Imaging, n. d.). Die Kombination dieser Parameter bildet kein bestimmtes Originalkameramodell vollständig nach. Eine reale Linsenverzeichnung wird nicht nachgebildet.

Die projektweite Nah-Clipping-Grenze ist in Unreal Engine mit `NearClipPlane=1.0` auf 1 cm eingestellt. Dadurch bleiben nahe Karosserieflächen sichtbar, die bei einer größeren Nah-Clipping-Distanz abgeschnitten würden. Die Einstellung beeinflusst die Darstellung und verändert nicht die Montagepositionen der Kameras.

6.4.2 Radar und ergänzender LiDAR

Die beiden Radare liegen vorne mittig übereinander (Zeichnung auf S. 46). 30 und 80 m entsprechen den oberen Grenzen der im Clevon-Datenblatt dargestellten Kurz- und Mittelbereiche (Clevon, n. d. b, S. 2). Sichtfelder und ein festes Intervall von 55 ms sind an smartmicro DRVEGRD 169 beziehungsweise 152 angelehnt (smartmicro, n. d.). Die Kombination ist kein Nachbau eines identifizierten Clevon-Radars. Je Sensor werden 5000 Strahlen/s angesetzt. CARLA liefert geometrische Treffer mit radialer Geschwindigkeit, keine vollständige FMCW-Signalkette oder Objektverfolgung (CARLA Team, n. d. e).

Das LiDAR-Profil orientiert sich am Ouster OS1 Rev. 05 mit 64 Kanälen, 360° horizontalem Sichtfeld und 10 Hz Rotation (Ouster, 2021). 655360 Strahlen/s entsprechen nominal 1024 horizontalen Abtastungen je Kanal und Umdrehung. Die Reichweitengrenze von 120 m ist keine garantierte reale Detektionsreichweite für beliebige Oberflächen. Zur Verringerung von Eigentreffern liegt der virtuelle Ursprung bei z=1,72 m und damit 13 cm über dem modellierten Gehäuseursprung. Dieser Versatz ist eine explizite Vereinfachung.

6.4.3 GNSS, IMU und Messraten

GNSS und IMU sind im Herstellerproduktblatt nicht ausgewiesen. Sie werden als für die autonome Navigation notwendige Grundausstattung angenommen. Die GNSS-Rate von 5 Hz orientiert sich an einer Betriebsart des u-blox ZED-F9P-05B, ohne RTK-Korrekturen oder RTK-Genauigkeit zu simulieren (u-blox, n. d., S. 4). 100 Hz dienen als IMU-Taktreferenz aus der SMI230-Dokumentation (Bosch, n. d.) und werden als Sollrate angesetzt. Der virtuelle GNSS-Sensor ist an der Position des modellierten Antennengehäuses angebracht, die IMU am Fahrzeugursprung. Rauschen und Bias sind deaktiviert. Tatsächliche Raten hängen von den Simulationsschritten ab; insbesondere sind 100 Hz IMU-Ausgabe im asynchronen Betrieb nicht zugesichert.

6.5 Lichtfunktionen und Ladeklappe

6.5.1 Leuchtflächen und Ausleuchtung

Die Lichtdarstellung kombiniert emissive Materialflächen mit zusätzlichen, am Fahrzeug angebrachten Lichtquellen. Die Materialemission stellt die leuchtenden Oberflächen dar; die zusätzlichen Lichtquellen ermöglichen eine gezielte Ausleuchtung der Umgebung. Beide werden aus demselben Funktionszustand angesteuert. Das der Komponente `Clevon_Lights` zugewiesene Mesh unterscheidet Rück-/Bremslicht, Blinker, Positionslicht, Frontlicht und Rückfahrlicht über separate Materialslots.

Beim `BeginPlay` werden dynamische Materialinstanzen erzeugt und unter anderem als `MID_TailBrake`, `MID_Reverse`, `MID_Position` und `MID_Headlight` gespeichert. Der skalare Parameter `EmissionStrength` multipliziert die Leuchtfarbe vor dem Materialeingang `Emissive Color`. Dadurch lässt sich die Emission zur Laufzeit zwischen dem ausgeschalteten Zustand und unterschiedlichen Helligkeitsstufen umschalten, ohne das Grundmaterial auszutauschen.

6.5.2 Verarbeitung der CARLA-Lichtzustände

Bei einer Zustandsänderung wird `Refresh Light State` ausgeführt. `Break VehicleLightState` stellt die einzelnen Lichtsignale bereit. Über `Select Float` werden die zugehörigen Emissions- beziehungsweise Lichtintensitäten ausgewählt. `Set Scalar Parameter Value` aktualisiert die dynamische Materialinstanz, während `Set Intensity` die zusätzliche Lichtquelle ansteuert. Für das gemeinsame Rück-/Bremslicht besitzt Bremsen Vorrang vor dem Positionslicht. Das Rückfahrlicht folgt dem Reverse-Signal. Fernlicht besitzt bei der Auswahl der Frontlichtstufe Vorrang vor Abblendlicht.

Die gesamte weiße Rückfahrlichtfläche ist dem Rückfahrlichtmaterial zugeordnet. Damit leuchtet neben den punktförmigen LED-Elementen auch die vorgesehene weiße Grundfläche. Die Kennzeichenbeleuchtung ist an den Positionslichtzustand gekoppelt.

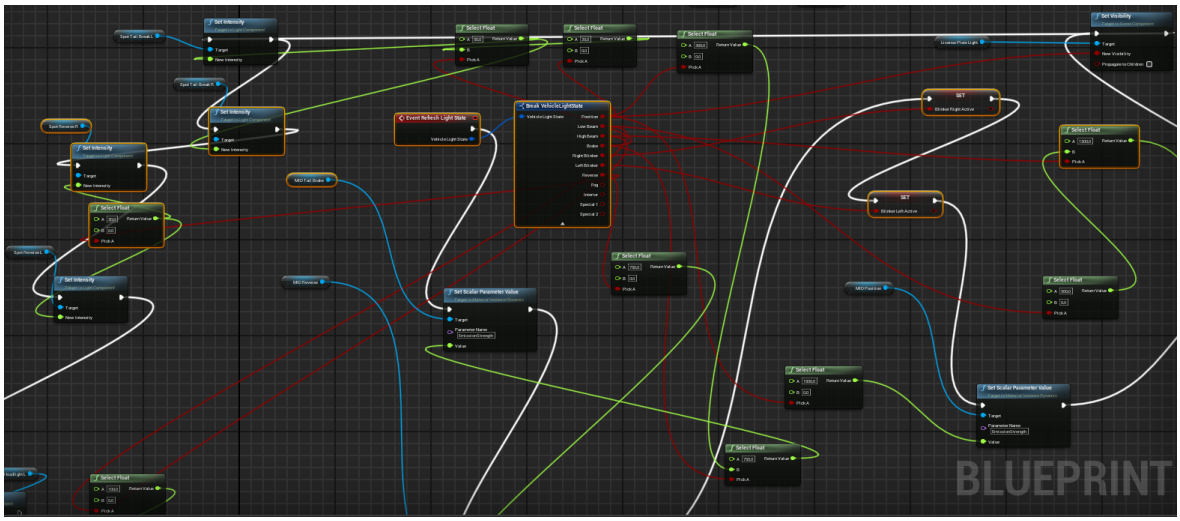


Abbildung 6.10: Auschnitt Ansteuerung von Leuchtmaterial und Lichtquelle aus dem CARLA-Lichtzustand. Eigener Screenshot aus Unreal.

6.5.3 Blinker und eingestellte Lichtintensitäten

`BlinkerLeftActive` und `BlinkerRightActive` speichern die Seitenfreigaben. Ein wiederholender Timer ruft im Abstand von 0,5 s `UpdateBlinkers` auf und wechselt `BlinkPhase`. Pro Seite wird die Freigabe logisch mit der Phase verknüpft. Dieses Ergebnis schaltet sowohl die Materialemission als auch die vorderen, seitlichen und hinteren Blinker-Lichtquellen. Nominal ergeben sich 0,5 s ein und 0,5 s aus. Bei gleichzeitiger Freigabe beider Seiten entsteht durch die gemeinsame Phase ein synchrones Warnblinken.

Die zusätzlichen Lichtquellen sind visuell abgestimmt. Für Abblend- und Fernlicht werden dieselben Frontstrahler mit 100 beziehungsweise 200 in der eingestellten Intensitätseinheit Lumen verwendet. Die Werte für Rücklicht, Bremslicht und Rückfahrlicht betragen 20, 50 und 35; für die Blinker wurde 1000 eingestellt. Im ausgeschalteten Zustand ist die Intensität null. Diese Werte beschreiben die Unreal-Konfiguration und keine gemessenen Lichtströme des Originalfahrzeugs.



Abbildung 6.11: Lichtfunktionen am integrierten Clevon in CARLA. Eigene Aufnahme.

6.5.4 Ladeklappe

Die CargoBox-Klappe ist als bewegliche Komponente ausgeführt. Der Constraint `CargoDoorConstraint` verbindet `CargoDoor` mit dem Fahrzeug-Mesh. Als Körperbezug des Skeletal Meshs ist `Vehicle_Base` eingetragen. Eine freigegebene Winkelbewegung bildet das Scharnier ab, während die übrigen Winkelbewegungen gesperrt sind.

Der CARLA-Türbefehl öffnet die Klappe physikalisch nach außen und oben. Beim Schließen wird sie unmittelbar in die geschlossene Lage zurückgesetzt. Die Öffnungs- und Schließfunktion wurde unter CARLA 0.10.0 erprobt. Ein exakt vermessener Öffnungswinkel von 90° und eine geregelte Schließtrajektorie sind nicht Bestandteil der Umsetzung.

Abbildung 6.12 zeigt die Modellgeometrie mit geöffneter Ladeklappe und freigegebener Ladeöffnung.



Abbildung 6.12: Modellansicht des Clevon mit geöffneter CargoBox-Ladeklappe. Eigene Darstellung.

7 ROS-2-Anbindung

Dieses Kapitel beschreibt, wie die Sensorik und die Steuerung von Clevon und Starship über ROS 2 bereitgestellt werden. Beide Fahrzeuge verwenden auf der gemeinsamen Plattform (CARLA 0.10.0, ROS 2 Jazzy, Ubuntu 24.04.2 LTS) denselben Aufbau; fahrzeugspezifisch sind nur die Sensorprofile und der Fahrzeugname.

7.1 Architektur

Die Grundbegriffe Topics, QoS und TF sowie das Zusammenspiel von Simulationstakt und Übertragungsrate sind in Abschnitt 3.2.1 beschrieben. CARLA 0.10.0 enthält eine eingebaute ROS-2-Schnittstelle, die mit dem Startparameter `--ros2` aktiviert wird (CARLA Team, n. d. d). Der Simulator veröffentlicht die Daten der angebrachten Sensoren dann selbst über DDS, ohne separate Bridge. Ein Sensor wird veröffentlicht, sobald er das Attribut `ros_name` trägt; sein Topic lautet `/carla/<fahrzeug>/<ros_name>/...`. Zusätzlich stellt CARLA die Simulationszeit auf `/clock` und die Lage der Sensoren relativ zum Fahrzeug auf `/tf` bereit.

Die Anbindung besteht aus drei Teilen (Abbildung 7.1):

1. **CARLA-Server mit eingebautem ROS 2:** veröffentlicht Kameras, Tiefenkameras, Radar, LiDAR, GNSS und IMU und empfängt Fahrbefehle.
2. **Sensorskript** (`<fahrzeug>_ros_sensors.py`): bringt die Sensoren aus den JSON-Profilen am Fahrzeug an und versieht sie mit `ros_name`. Es verarbeitet selbst keine Daten.
3. **Ergänzungsknoten** (`<fahrzeug>_ros_extras.py`): ein rclpy-Knoten, der veröffentlicht, was CARLA nicht selbst bereitstellt: die Fahrzeuglage und beim Starship die Ultraschallwerte.

Der Clevon besitzt zusätzlich eine bereits vorhandene ROS-2-Bridge für CARLA 0.9.16 unter Windows/WSL 2 mit ROS 2 Humble. Sie liest Sensordaten über die Python-API und veröffentlicht sie selbst. Auf der neuen Plattform ersetzt die native Anbindung diese Bridge; beide dürfen nicht gleichzeitig betrieben werden, da sonst jeder Sensor doppelt angebracht wird.

7.2 Verknüpfung mit den Sensorskripten

Grundlage sind dieselben JSON-Profile, die auch die Anzeigeprogramme verwenden (Kapitel 4). Jedes Profil enthält je Sensor Blueprint, Montageposition, Attribute und einen eindeutigen `role_name`. Das Sensorskript übernimmt diesen Namen als `ros_name`. Ein Sensor trägt damit in Anzeige, Messdatei und ROS 2 denselben Namen, und Änderungen an Position oder Attributen gelten für alle Anwendungen.

Die Verbindung zu CARLA, die Profilprüfung, das Anbringen und das Entfernen der Sensoren sowie die Ermittlung des ROS-Fahrzeugnamens liegen je Fahrzeug

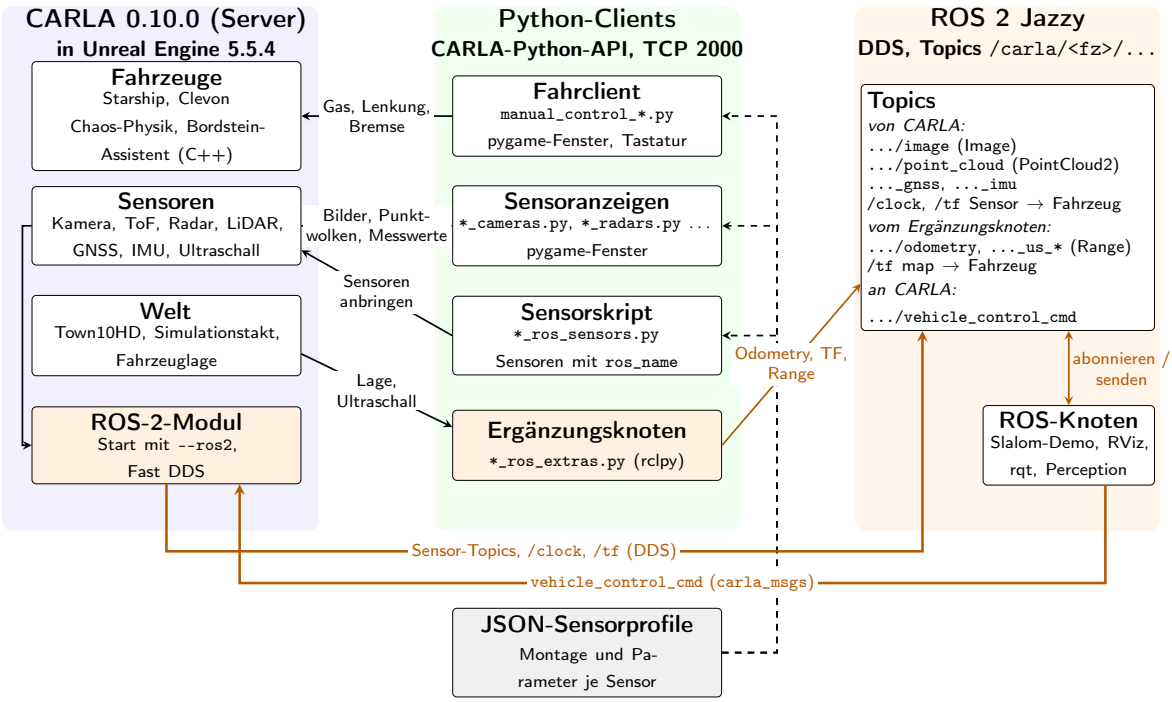


Abbildung 7.1: Architektur des Aufbaus. Python-Clients steuern CARLA über die CARLA-Python-API (TCP) und zeigen Sensordaten in pygame-Fenstern an. Mit `--ros2` veröffentlicht CARLA die Sensordaten selbst über DDS; der Ergänzungsknoten liefert Fahrzeuglage und Ultraschall. Fahrbefehle erreichen CARLA über `vehicle_control_cmd`. Eigene Darstellung.

im gemeinsamen Modul (`clevon_sensor_common.py`, `starship_sensor_common.py`). Beide Module haben dieselbe Schnittstelle, sodass Sensorskript und Ergänzungsknoten für beide Fahrzeuge gleich aufgebaut sind.

Tabelle 7.1: Sensorgruppen und Profile je Fahrzeug

Gruppe	Clevon	Starship
`nav`	GNSS, IMU	GNSS, IMU
`radar`	2 Radare	4 Radare
`lidar`	1 LiDAR	–
`tof`	–	3 Tiefenkameras (ToF)
`cameras`	6 RGB-Kameras	9 RGB-Kameras
Ergänzungsknoten	–	8 Ultraschall
Summe	11 Sensoren	26 Sensoren

Die Gruppen lassen sich einzeln starten. Das erlaubt, rechenintensive Sensoren wie Kameras wegzulassen oder, beim Clevon, mit reduzierter Auflösung anzubringen. Sensoren, die bereits am Fahrzeug hängen, werden übersprungen: Der Fahrclient bringt GNSS und IMU schon mit `ros_name` an, das Sensorskript ergänzt dann nur die übrigen Gruppen.

7.3 Namensgebung und Koordinatensysteme

Ohne `ros_name` vergibt CARLA den Fahrzeugnamen `vehicle<ID>`, der sich bei jedem Spawn ändert. Fahrclient und Spawn-Skript setzen deshalb einen festen Namen (`clevon` bzw. `starship`). Alle Topics liegen damit unter einem stabilen Präfix, etwa `/carla/starship/starship_cam_front/image`.

Die Umrechnung von CARLA- in ROS-Koordinaten (REP 103, Kapitel 4) führt CARLA für seine eigenen Topics selbst aus; der Ergänzungsknoten verwendet dieselbe Umrechnung (Foote & Purvis, n. d.). Der TF-Baum ist nach REP 105 aufgebaut (Meeussen, n. d.): `map` → Fahrzeug stammt aus dem Ergänzungsknoten, Fahrzeug → Sensor aus CARLA bzw. für den Ultraschall aus dem Ergänzungsknoten. Damit lässt sich jede Messung in Kartenkoordinaten transformieren.

7.4 Ergänzungsknoten

Die eingebaute Schnittstelle weist zwei Lücken auf, die der Ergänzungsknoten schließt:

- **Fahrzeuglage:** CARLA veröffentlicht weder Odometrie noch die Transformation `map` → Fahrzeug. Der Knoten liest Lage und Geschwindigkeit in jedem Simulationsschritt aus und veröffentlicht sie als `nav_msgs/Odometry` auf `/carla/<fahrzeug>/odometry` sowie auf `/tf`. Es handelt sich um Referenzwerte der Simulation, nicht um eine Lokalisierung.
- **Ultraschall (Starship):** Für den Hinderniserkennungssensor, der den Ultraschall nachbildet, enthält CARLA keinen ROS-2-Publisher. Der Knoten bringt die acht Sensoren aus dem Ultraschallprofil selbst an und veröffentlicht ihre Entfernungen als `sensor_msgs/Range`. Ohne Treffer wird nach REP 117 der Wert $+\infty$ gesendet (Rockey, n. d.).

Alle Nachrichten des Knotens tragen wie die Topics von CARLA die Simulationszeit als Zeitstempel. Die Ausgaberate beträgt standardmäßig 20 Hz.

7.5 Fahrzeugsteuerung über ROS 2

Für Fahrzeuge mit der Rolle `hero` legt CARLA einen Subscriber auf `/carla/<fahrzeug>/vehicle_control_cmd` an. Er erwartet Nachrichten vom Typ `carla_msgs/CarlaEgoVehicleControl` mit Gas, Lenkung, Bremse, Handbremse, Rückwärtsgang und Gangwahl. Da `carla_msgs` für ROS 2 Jazzy nicht als Paket verfügbar ist, enthält das Projekt einen minimalen Arbeitsbereich, der nur diese Nachricht mit den von CARLA erwarteten Feldern definiert.

Gesteuert wird ein Fahrzeug, das ohne Fahrclient gespawnt wurde, da ein Fahrclient die Steuerung in jedem Takt überschreibt. Damit kann jeder ROS-2-Knoten, etwa ein Planer oder ein Teleoperationsknoten, das Fahrzeug steuern. Zu beachten ist:

- Die Lenkung folgt der CARLA-Konvention (positiv = rechts), nicht der ROS-Winkelkonvention.

- Der Subscriber arbeitet mit *best effort*. Ein neu gestarteter Publisher braucht 1–3 s zur DDS-Erkennung; vorher gesendete Nachrichten gehen verloren (Open Robotics, n. d. a). Einzelbefehle werden deshalb erst nach erfolgter Verbindung gesendet.

Tabelle 7.2: Steuerung über ROS 2 (Town10HD, ohne Fahrclient)

Befehl	Clevon	Starship
Gas 0,5 bzw. 0,7	14,5 km/h	6,1 km/h (Höchstgeschw.)
Volllenkung	Rechtskurve	Linkskurve, 5,4 km/h
Bremse 1,0	Stillstand	Stillstand in 1 s

7.6 Betrieb

Für beide Fahrzeuge gilt dieselbe Startreihenfolge. Fahrzeugspezifisch sind nur die Skriptnamen:

1. CARLA mit `--ros2` starten, mit Fenster oder ohne Darstellung (`-RenderOffScreen`, spart Grafikspeicher).
2. Fahrzeug spawnen: über den Fahrclient zum manuellen Fahren oder über das Spawn-Skript zum Steuern über ROS 2.
3. Sensorskript starten, wahlweise mit einzelnen Gruppen.
4. Ergänzungsknoten starten.
5. Optional: Nachführkamera, um das Fahrzeug im CARLA-Fenster zu verfolgen.

Die Daten lassen sich mit den Standardwerkzeugen von ROS 2 prüfen, etwa mit `ros2 topic hz` für die Rate, `ros2 topic echo` für den Inhalt und `tf2_echo` für die TF-Kette. Sensorskript und Ergänzungsknoten entfernen ihre Sensoren beim Beenden wieder.

7.7 Übertragungsraten

Wie in Abschnitt 3.2.1 beschrieben, begrenzt der Simulationstakt die erreichbare Rate jedes Sensors. Tabelle 7.3 zeigt die gemessenen Raten mit allen Sensoren gleichzeitig im asynchronen Betrieb.

Sensoren mit niedriger Sollrate erreichen diese zuverlässig. Sensoren mit hoher Sollrate, insbesondere die IMU, sind durch den Simulationstakt begrenzt. Beim Clevon senken hochauflösende Kameras den Takt so stark, dass in Echtzeit nur etwa 7 Bilder/s erreicht werden; der Grafikspeicher steigt dabei auf 12 GB. Der Starship kommt mit allen 26 Sensoren auf 11 GB.

Tabelle 7.3: Gemessene Übertragungsraten (Town10HD, asynchron, alle Sensoren aktiv)

Fahrzeug	Sensor (Nachrichtentyp)	Soll	Ist
Starship	9 Kameras (`Image`)	4 Hz	3,8–4,2 Hz
	3 ToF (`Image`)	10 Hz	10 Hz
	4 Radare (`PointCloud2`)	20 Hz	19,8–20,2 Hz
	8 Ultraschall (`Range`)	20 Hz	20 Hz
	GNSS (`NavSatFix`)	5 Hz	5 Hz
	IMU (`Imu`)	100 Hz	28–60 Hz[a]
	Odometrie (`Odometry`)	20 Hz	20 Hz
Clevon	6 Kameras (`Image`), 640×360	10 Hz	10 Hz
	6 Kameras, 1280×720	20 Hz	11,3 Hz[a]
	6 Kameras, 1920×1080	30 Hz	ca. 7 Hz[a]
	2 Radare (`PointCloud2`)	18,2 Hz	18,0–18,6 Hz
	LiDAR (`PointCloud2`)	je Schritt	39 Hz
	GNSS (`NavSatFix`)	5 Hz	5 Hz
	IMU (`Imu`)	100 Hz	38 Hz[a]
	Odometrie (`Odometry`)	20 Hz	20 Hz

[a] durch den Simulationstakt begrenzt

7.8 Grenzen

- Die Tiefenkameras senden das Bild in der CARLA-Farbkodierung (`bgra8`), nicht in Metern (`32FC1`); Empfänger müssen umrechnen.
- Die Odometrie liefert Referenzwerte der Simulation ohne Rauschen.
- Ein Not-Halt bei ausbleibenden Fahrbefehlen fehlt.
- Der synchrone Betrieb, der für jeden Simulationsschritt vollständige Messungen liefert, wurde nicht getestet.
- Getestet ist ausschließlich die native Anbindung unter Linux; die frühere Clevon-Bridge unter Windows/WSL 2 wurde gegen CARLA 0.10.0 nicht geprüft.

8 Validierung und Ergebnisse

8.1 Prüfumfang

Die Ergebnisse folgen den vier Prüfbereichen aus Kapitel 2. Da keine Messdaten der realen Fahrzeuge vorliegen, ist die Fahrphysik plausibilisiert und funktional nachgewiesen, nicht gegen das Original validiert. Skripte und Rohdaten liegen im Projekt-Repository (Rauch & Le, 2026).

8.2 Geometrie

Die Maße beider Modelle weichen in Breite und Höhe um weniger als 5 % von den Referenzangaben ab; die Länge des Starship ist designbedingt um 16,7 % kürzer (Tabellen 6.1 und 5.1). Import, Skalierung und die zur Laufzeit zurückgelesenen Fahrzeugparameter stimmen mit den Sollwerten überein.

8.3 Fahrphysik

Belegt sind beim Clevon das 6-kW-Drehmomentmodell, beim Starship Masse und Höchstgeschwindigkeit. Die übrigen Parameter wurden in eigenen Fahrten iterativ abgestimmt (Tabelle 8.1).

Tabelle 8.1: Fahrphysikalische Ergebnisse in Town10HD; eigene Messung bzw. Beobachtung.

Prüfung	Clevon	Starship
Höchstgeschwindigkeit	um 50 km/h (berechnet 50,3)	6,27 km/h (berechnet 6,1)
Beschleunigung	ca. 4 s auf 20 km/h	90 % von $v_{\max}$ nach 0,8 s
Bremsweg / Wenderadius	nicht gemessen	0,22 m / 2,2–2,4 m
Bordstein	Eintauchen durch Spherecast behoben	hoch 10/12, runter 8/8

Ohne Bordstein-Assistent gelang dem Starship kein Auffahren (0/3). Die Bordsteinmessung erfolgte mit Raycast-Radabfrage; mit Spherecast liegt bisher nur eine Stichprobe vor.

8.4 Sensorik und ROS 2

Alle vorgesehenen Sensoren sind umgesetzt: elf beim Clevon, 26 beim Starship. Die Anzeigeprogramme empfangen und speichern die Daten aller Sensortypen; beim Starship wurden Ultraschall, ToF und Radar zusätzlich auf plausible Messwerte geprüft. Die erreichbaren Raten sind durch den Simulationstakt begrenzt; die IMU erreicht ihre 100 Hz bei keinem Fahrzeug (Tabelle 7.3).

Über ROS 2 stehen für beide Fahrzeuge alle Sensordaten, Odometrie und TF-Kette zur Verfügung. Anfahren, Kurvenfahrt und Bremsen per ROS-Befehl wurden

erfolgreich geprüft (Tabelle 7.2); beim Clevon zusätzlich Licht- und Klappenbefehle über die frühere Bridge. Die Prüfungen belegen die Ausführbarkeit der Schnittstelle, nicht die Güte eines Fahrreglers.

8.5 Zusammenfassung und Grenzen

Tabelle 8.2: Nachweise je Prüfbereich.

Prüfbereich	Clevon	Starship
Geometrie	Abweichung unter 5 %	Länge −16,7 %, sonst unter 5 %
Fahrphysik	Drehmomentmodell belegt, Fahrverhalten beobachtet	Geschwindigkeit, Bremsweg, Wenderadius, Bordstein gemessen
Sensorik	11 Sensoren, Daten empfangen	26 Sensoren, Daten empfangen und plausibilisiert
ROS 2	Daten, Fahr-, Licht- und Klappenbefehle	Daten und Fahrbefehle

Nicht vorhanden sind Vergleichsmessungen an realen Fahrzeugen, Messreihen im synchronen Betrieb und ein Not-Halt in der nativen ROS-2-Anbindung. Diese Grenzen werden nicht als erfolgreiche Tests gewertet.

9 Diskussion und Grenzen

Beide Fahrzeuge sind als funktionsfähige Simulationsmodelle umgesetzt. Dieses Kapitel ordnet ein, wo die Modelle bewusst oder zwangsläufig vom Original abweichen und welche Aussagen daraus folgen.

9.1 Gemeinsame Grenzen

Ausrichtung, Aussehen, Position und Ausführung der Sensoren wurden so weit wie möglich aus Literatur, Herstellerangaben und Bildern abgeleitet. Wo Angaben fehlen, sind die Werte begründete Annahmen. Referenzprodukte begründen marktübliche Messraten und Sichtfelder, belegen aber nicht die Identität mit den tatsächlich verbauten Komponenten. Die CARLA-Sensoren sind zudem idealisiert: Linsenverzeichnung, Mehrwegeffekte und materialabhängige Reflexionen werden nicht oder nur vereinfacht abgebildet; ToF und Ultraschall sind Nachbildungen über verwandte Sensortypen. Die Übertragbarkeit auf reale Messfehler und Abschattungen ist damit begrenzt.

Die ROS-Tests belegen Datenempfang und ausführbare Stellbefehle. Eine externe Anwendung muss dennoch Topic-Namen, QoS, Koordinaten und den Fahrzeugbefehlstyp berücksichtigen. Die Ratenbegrenzung durch den Simulationstakt zeigt, dass eine funktionierende Verbindung nicht mit garantierter Echtzeitfähigkeit gleichzusetzen ist. Für quantitative Wahrnehmungsvergleiche wären reproduzierbare Szenarien, ein synchroner Betrieb und dokumentierte Datenraten erforderlich.

9.2 Starship

Lenkung. Das Modell lenkt wie ein Pkw über die Vorderräder mit 30° Lenkwinkel; der Wenderadius beträgt 2,2 bis 2,4 m. Eine Panzerlenkung über unterschiedliche Antriebsmomente der Seiten oder eine getrennte Ansteuerung der drei Achsen ist nicht umgesetzt, da die CARLA-Fahrzeugsteuerung nur gemeinsame Werte für Gas, Lenkung und Bremse überträgt. Ein Drehen auf der Stelle ist daher nicht möglich. Ein Lösungsweg über Antriebsmomente je Rad ist in Abschnitt 5.5 beschrieben.

Bordstein. Das Original überwindet Bordsteine mechanisch über Kipphebel zwischen Mittel- und Hinterrad. Chaos Vehicles kennt keine Gelenke zwischen Rädern; das Modell ersetzt den Mechanismus durch geregelte Hubkräfte. Es bildet damit die Wirkung nach, nicht den Bewegungsablauf: Der starre Aufbau kippt beim Auffahren um rund 30°, und etwa eine von sechs Auffahrten bleibt hängen.

Autopilot und Gehwegnavigation. Der CARLA-Autopilot behandelt den Bot als Fahrzeug und führt ihn auf der Fahrbahn statt auf dem Gehweg. Die Trajektorien der CARLA-Fußgänger sind nicht ohne Weiteres nutzbar, da deren Navigation nur für Fußgänger-Actors vorgesehen ist. Eine eigene Bahnführung entlang der Gehwege ist machbar, aber nicht umgesetzt (Abschnitt 5.5).

Aussehen. Das Modell ist eine vereinfachte Nachbildung im eigenen HKA-Design. Mangels CAD-Daten wurden Grundform und Proportionen übernommen, Details vereinfacht; die Länge ist um 16,7 % kürzer als das Original. Statt der Fahne trägt die Antenne ein rotes Licht. Das eigene Design kennzeichnet das Modell als Projektfahrzeug und vermeidet die Nachbildung der Marke des Herstellers. Für die Simulation ist das Aussehen nachrangig, solange Maße, Kollisionskörper und Sensorpositionen stimmen.

9.3 Clevon

Der Clevon trägt zusätzlich einen LiDAR auf dem Dach. Diese Abweichung vom Original ist gewollt, damit auch dieser Sensortyp in der Simulation bereitsteht und weiterverarbeitet werden kann. Der LiDAR ist als optionale Variante gekennzeichnet; sein Ursprung liegt zur Vermeidung von Eigentreffern über dem modellierten Gehäuse.

Die Maßabweichungen und die angenäherten Fahrwerksparameter begrenzen die Aussage, das reale Fahrzeug exakt abzubilden. Unterschiedliche Massenangaben in den Herstellerunterlagen zeigen zudem, dass Fahrzeugkonfiguration und Quellenstand berücksichtigt werden müssen. Kameras ohne Verzeichnung und deaktiviertes Rauschen vereinfachen die Auswertung, reduzieren aber die Realitätsnähe.

9.4 Vergleich der Fahrzeugtypen

Beide Modelle decken zwei unterschiedliche Einsatzfälle ab: den Clevon als Straßenroboter mit bis zu 50 km/h und den Starship als Gehwegroboter mit Schrittgeschwindigkeit. Der Clevon fügt sich in die Fahrspurlogik von CARLA ein und kann grundsätzlich den Autopiloten nutzen; der Starship benötigt für seinen eigentlichen Einsatzraum eine eigene Bahnführung und für Bordsteine eine Zusatzkomponente. Beide stellen ihre Sensordaten und Fahrbefehle über dieselbe ROS-2-Struktur bereit, sodass externe Anwendungen für beide Fahrzeuge gleich angebunden werden können.

10 Fazit und Ausblick

Die leitende Frage, wie sich zwei unterschiedliche Lieferroboter als fahrfähige, sensorisch ausgestattete CARLA-Modelle umsetzen und für externe ROS-2-Module bereitstellen lassen, ist für beide Fahrzeuge beantwortet. Bezogen auf die Projektaufgabe (Hochschule Karlsruhe, Institut für Energieeffiziente Mobilität, 2026) ergibt sich:

- **Gehwegroboter:** Der Starship ist als sechsrädriges Fahrzeug mit Allradantrieb, Schrittgeschwindigkeit und Bordstein-Assistent umgesetzt.
- **Straßenroboter:** Der Clevon ist mit Lichtfunktionen, Ladeklappe und 6-kW-Antriebsmodell umgesetzt.
- **Fahrphysik:** Beide Modelle laufen auf Chaos Vehicles unter CARLA 0.10.0. Belegbare Parameter stammen aus Herstellerangaben, die übrigen wurden in Fahrversuchen iterativ abgestimmt.
- **Sensorik:** Alle vorgesehenen Sensoren sind als JSON-Profile hinterlegt, aus Referenzen abgeleitet und über Anzeigeprogramme nutzbar: elf beim Clevon, 26 beim Starship.
- **ROS 2:** Beide Fahrzeuge stellen Sensordaten, Odometrie und Koordinatensysteme unter festen Topic-Namen bereit und nehmen Fahrbefehle entgegen. Die Schnittstelle ist für beide Fahrzeuge gleich aufgebaut und dokumentiert.
- **Funktionstests:** Geometrie, Fahrverhalten, Sensordaten und ROS-Steuerung wurden in CARLA geprüft; die Ergebnisse sind reproduzierbar dokumentiert.

Die Modelle bilden damit eine technische Grundlage, keine vollständige Replikation der realen Fahrzeuge. Ihre Grenzen liegen vor allem in fehlenden Vergleichsmessungen am Original, idealisierten Sensoren und der vereinfachten Lenk- und Kletterkinematik des Starship.

Für weiterführende Arbeiten ergeben sich folgende Schritte:

- Anbindung externer Perception-Modelle über die ROS-2-Schnittstelle und Analyse möglicher Sicherheitsrisiken, etwa manipulierter Sensordaten oder Fahrbefehle.
- Synchroner Simulationsbetrieb für vollständige, reproduzierbare Datensätze.
- Not-Halt bei ausbleibenden Fahrbefehlen in der nativen ROS-2-Anbindung.
- Gehwegnavigation und Panzerlenkung für den Starship (Abschnitt 5.5).
- Abgleich von Fahrverhalten und Sensordaten mit Messungen an realen Fahrzeugen.

Literaturverzeichnis

Bosch. (n. d.). *Bosch SMI230 IIO Driver*. Verfügbar 10. Oktober 2026 unter https://boschmemssolutions.github.io/iio/bosch_smi230_IIO.html

CARLA Team. (n. d. a). *Bounding boxes*. CARLA Simulator. Verfügbar 21. Oktober 2026 unter https://carla.readthedocs.io/en/latest/tuto_G_bounding_boxes/

CARLA Team. (n. d. b). *Content authoring – vehicles* [CARLA 0.10.0]. Verfügbar 22. Oktober 2026 unter https://carla-ue5.readthedocs.io/en/latest/tuto_content_authoring_vehicles/

CARLA Team. (n. d. c). *Python API* [CARLA 0.10.0]. Verfügbar 22. Oktober 2026 unter https://carla-ue5.readthedocs.io/en/latest/python_api/

CARLA Team. (n. d. d). *ROS2* [CARLA 0.10.0]. Verfügbar 22. Oktober 2026 unter https://carla-ue5.readthedocs.io/en/latest/ros2_native/

CARLA Team. (n. d. e). *Sensors reference* [CARLA 0.10.0]. Verfügbar 22. Oktober 2026 unter https://carla-ue5.readthedocs.io/en/latest/ref_sensors/

CARLA Team. (n. d. f). *Synchrony and time-step* [CARLA 0.10.0]. Verfügbar 22. Oktober 2026 unter https://carla-ue5.readthedocs.io/en/latest/adv_synchrony_timestep/

Clevon. (n. d. a). *CargoBox: A configurable all-round delivery solution* (Produktdatenblatt, ohne ausgewiesenes Jahr. Lokale Quelle: Clevon-CargoBox-Product-sheet.pdf).

Clevon. (n. d. b). *CLEVON 1: The multi-purpose autonomous delivery vehicle* (Produktdatenblatt, metrische Einheiten, ohne ausgewiesenes Jahr. Lokale Quelle: Product-sheet-CLEVON-1-metric-units.pdf).

Clevon. (2023). *DHL Express Estonia and Clevon Deliver the Future of Last Mile Delivery Today* (Case Study) (Lokale Quelle: Clevon-DHL-case-study.pdf).

Dimensions.com. (n. d.). *Starship robot*. Verfügbar 22. Oktober 2026 unter https://dimensions.com/element/starship-robot

e-con Systems. (n. d.). *e-CAM20_CU0230_MOD: 2MP AR0230 Automotive Grade HDR Camera Module*. Verfügbar 10. Oktober 2026 unter https://www.e-consystems.com/2mp-ar0230-hdr-camera-module-for-automotive.asp

Foote, T., & Purvis, M. (n. d.). *REP 103: Standard units of measure and coordinate conventions*. Verfügbar 21. Oktober 2026 unter https://www.ros.org/reps/rep-0103.html

Gallant, K. (n. d.). *Model a 3d Car | part 3 | intro to modelling (Blender tutorial)* [Video-Tutorial]. YouTube. Verfügbar 10. Oktober 2026 unter https://www.youtube.com/watch?v=Wj4LbDuQAmI

Gryd CG. (n. d.). *Blender Advanced Car Modeling Master Class (Part 1)* [Video-Tutorial; Kanalname: Gryd CG "Toby"]. YouTube. Verfügbar 10. Oktober 2026 unter https://www.youtube.com/watch?v=zhzRdTciAIU

Heinla, A., Volkov, K.-R., Roberts, L., & Mandre, I. (2018, 6. September). *Mobile robot system and method for autonomous localization using straight lines extracted from visual images* (US-Pat. Nr. US 2018/0253107 A1). Verfügbar 22. Oktober 2026 unter https://patents.google.com/patent/US20180253107A1/en

Hochschule Karlsruhe, Institut für Energieeffiziente Mobilität. (2026). *Implementierung von Last-Mile-Delivery-Robotern in CARLA* (Projektaufgabe) (Projekt-Code 26ss_RK_Roboter_in_CARLA, Sommersemester 2026).

Infineon Technologies AG. (n. d.). *REAL3 ToF 3D image sensors*. Verfügbar 21. Oktober 2026 unter https://www.infineon.com/cms/en/product/sensor/tof-3d-image-sensors/

Leopard Imaging. (n. d.). *LI-AR0233-GMSL3-130H: Data Sheet*. Verfügbar 10. Oktober 2026 unter https://leopardimaging.com/wp-content/uploads/2023/11/LI-AR0233-GMSL3-130H_Datasheet.pdf

Liivik, I., Liivik, M., & Liivik, T. (2024, 11. Juni). *System and method for traversing vertical obstacles* (US-Pat. Nr. US 12005753 B2). Verfügbar 22. Oktober 2026 unter https://patents.google.com/patent/US12005753B2/en

MaxBotix. (n. d.). *Mb1242 i2cxl-maxsonar-ez4*. Verfügbar 22. Oktober 2026 unter https://maxbotix.com/products/mb1242

Meeussen, W. (n. d.). *REP 105: Coordinate frames for mobile platforms*. Verfügbar 21. Oktober 2026 unter https://www.ros.org/reps/rep-0105.html

MIT Technology Review. (2018, 30. Januar). *Why sidewalk delivery robots still need safety drivers, too*. Verfügbar 22. Oktober 2026 unter https://www.technologyreview.com/2018/01/30/145935/why-sidewalk-delivery-robots-still-need-safety-drivers-too/

Open Robotics. (n. d. a). *Quality of service settings: Ros 2 documentation: Jazzy*. Verfügbar 21. Oktober 2026 unter https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html

Open Robotics. (n. d. b). *Tf2: Ros 2 documentation: Jazzy*. Verfügbar 21. Oktober 2026 unter https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Tf2.html

Open Robotics. (n. d. c). *Topics: Ros 2 documentation: Jazzy*. Verfügbar 21. Oktober 2026 unter https://docs.ros.org/en/jazzy/Concepts/Basic/About-Topics.html

Ouster. (2021, 24. August). *OS1: Mid-Range High-Resolution Imaging Lidar* (Rev. 05, Firmware v2.1.x). Verfügbar 10. Oktober 2026 unter https://data.ouster.io/downloads/datasheets/datasheet-rev05-v2p1-os1.pdf

Rauch, R., & Le, A. (2026). *Last Mile Delivery CARLA: Modell- und Integrationsdaten* [Clevon und Starship, Branch merge/clevon-starship-ein-repo, Stand 22.10.2026; Messungen vom 9. bis 22. Oktober 2026]. Verfügbar 22. Oktober 2026 unter https://github.com/rari1014/last_mile-delivery-carla

renderZone. (2024, 6. September). *Car wheel pack* [3D-Modell, Modell-ID 5510560; Quelle des übernommenen Felgensatzes]. CGTrader. Verfügbar 10. Oktober 2026 unter https://www.cgtrader.com/free-3d-models/car/car/car-wheel-pack-9940fb32-5b32-41e6-b278-92ae864c9a56

Rockey, C. (n. d.). *REP 117: Informational distance measurements*. Verfügbar 22. Oktober 2026 unter https://www.ros.org/reps/rep-0117.html

smartmicro. (n. d.). *DRVEGRD Radar Sensor Line for Automotive Applications*. Verfügbar 10. Oktober 2026 unter https://www.smartmicro.com/automotive-radar-sensors/products/drvegrd-line/

TechInsights. (n. d.). *Infineon IRS2875C ToF image sensor*. Verfügbar 21. Oktober 2026 unter https://www.techinsights.com/ko/node/61619

Texas Instruments. (n. d.). *Iwr6843aop: Single-chip 60-ghz to 64-ghz intelligent mmwave sensor with integrated antenna on package*. Verfügbar 22. Oktober 2026 unter https://www.ti.com/product/iwr6843aop

u-blox. (n. d.). *ZED-F9P-05B: Data Sheet* (UBXDOC-963802114-12824, R02). Verfügbar 10. Oktober 2026 unter https://content.u-blox.com/sites/default/files/documents/ZED-F9P-05B_DataSheet_UBXDOC-963802114-12824.pdf

Wevolver. (n. d.). *Starship robot: Tech specs*. Verfügbar 22. Oktober 2026 unter https://wevolver.com/specs/starship-technologies-starship-robot

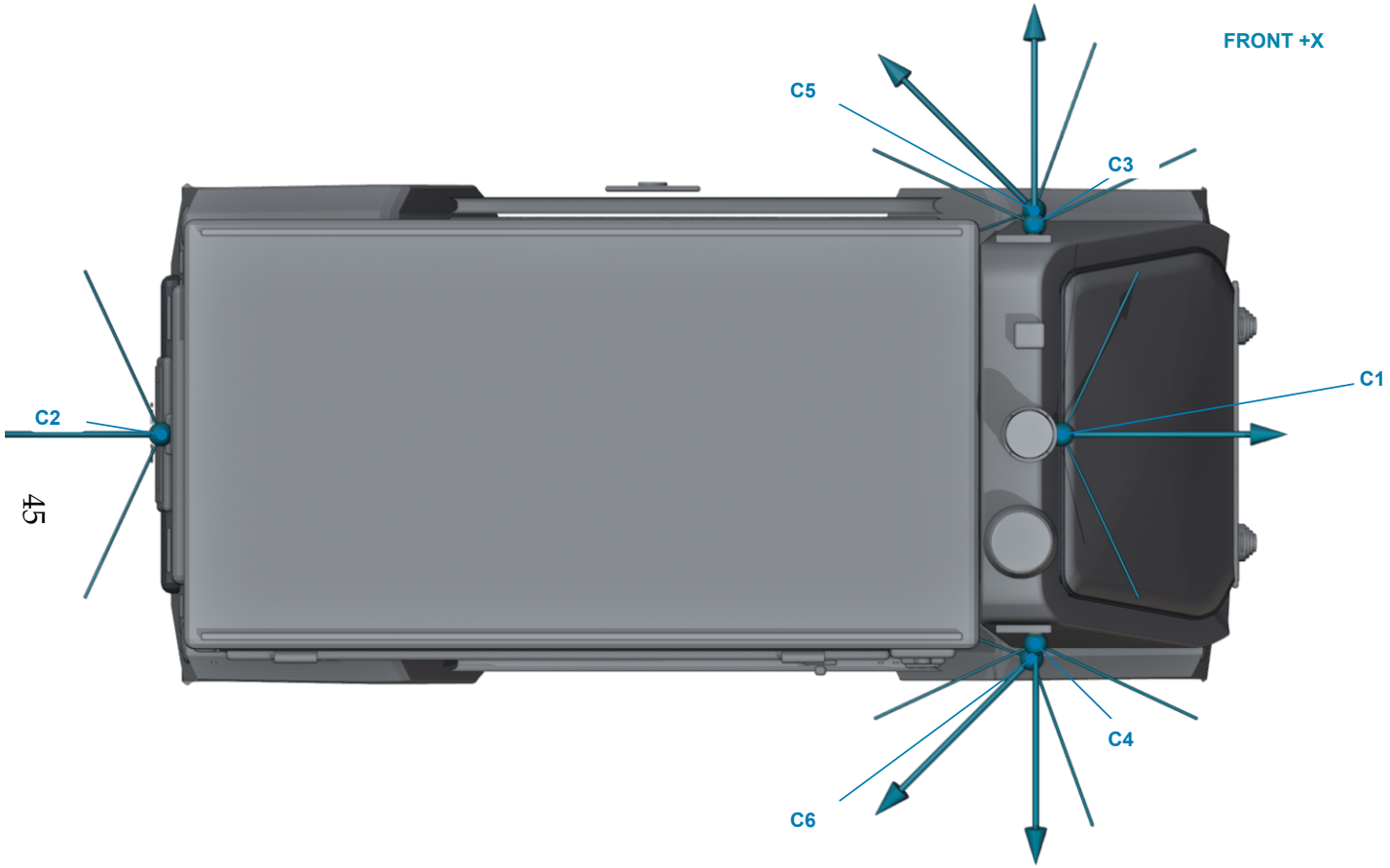
A Clevon: Sensorzeichnungen

Die folgenden Blätter dokumentieren die Anordnung der Kameras sowie von Radar, LiDAR und Navigationssensoren am Clevon-Modell. Quelle: eigene Geometrieauswertung aus den CARLA-Montagewerten.

Kameras: Position und Blickrichtung

Blaue Markierungen aus den gespeicherten CARLA-Montagewerten | C1 bis C6 | ungespiegelte Kamerabilder

DRAUFSICHT



45

ID	Position	Yaw
C1	Front unter LiDAR	0°
C2	Heck / Kennzeichen	180°
C3	Links oben	-90°
C4	Rechts oben	90°
C5	Links unten	-135°
C6	Rechts unten	135°

Blickwinkel
Alle Kameras: 130° horizontal.
Front und Heck: Pitch 0°.
Seitliche Kameras: Pitch -10°.
Untere Seitenkameras: 45° nach hinten gegenüber der seitlichen Richtung.

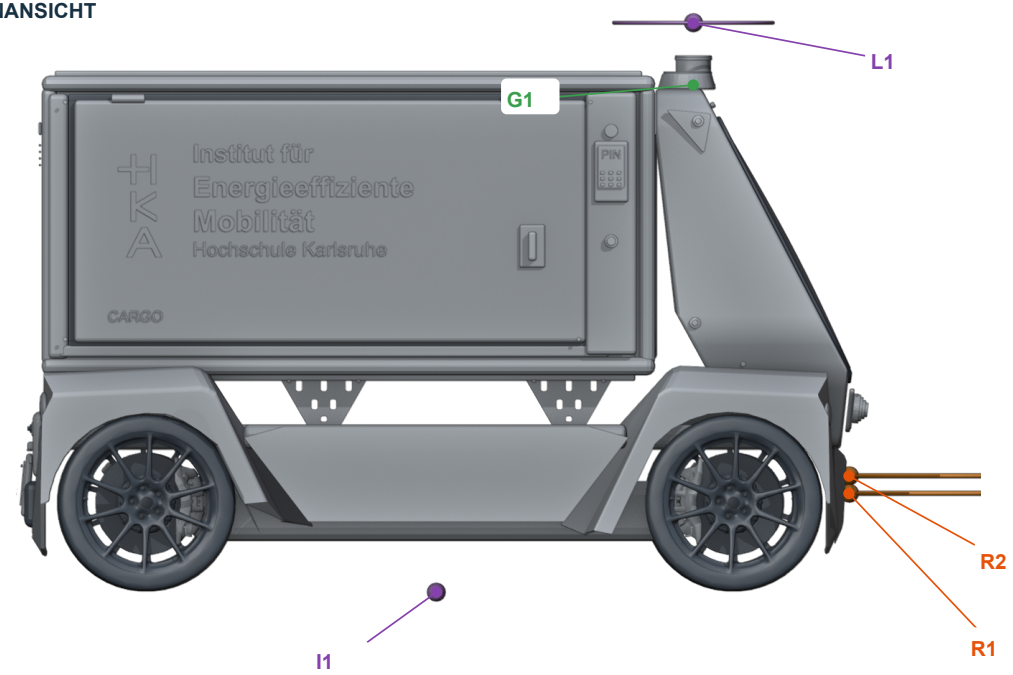
Darstellung
Pfeile zeigen die optische Achse. Kurze Randlinien zeigen den horizontalen Öffnungswinkel. Ihre Länge ist schematisch; sie bildet keine Erkennungsreichweite ab.

Einordnung: Positionen folgen den modellierten Kameragehäusen mit den dokumentierten Montagekorrekturen. Blickrichtungen und Optik sind Projektannahmen, keine vermessene OEM-Kalibrierung. Die Darstellung berechnet keine Abschattung durch die Karosserie und ist kein Nachweis einer lückenlosen 360°-Sicht.

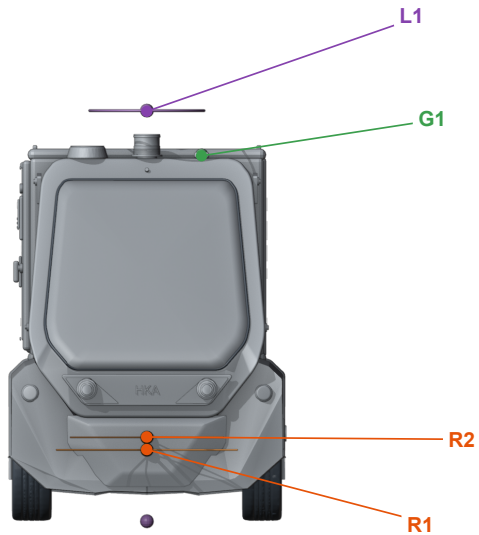
Radar, LiDAR und Navigation

Montagehöhen und virtuelle Sensorpunkte | Orange: Radar | Grün: GNSS | Violett: LiDAR und IMU

SEITENANSICHT



FRONTANSICHT



ID	Funktion	Z in m	Richtung	FoV horizontal	FoV vertikal
R1	Kurzbereich, 30 m	0,299	vorwärts	130°	15°
R2	Mittelbereich, 80 m	0,351	vorwärts	100°	20°
L1	LiDAR, ergänzend	1,720	umlaufend	360°	-22,5° bis +22,5°
G1	GNSS-Montagepunkt	1,533	keine Kameraachse	-	-
I1	IMU, Fahrzeugursprung	0,000	Fahrzeugachsen	-	-

LiDAR-Abweichung: Der virtuelle Messpunkt liegt bei Z = 1,720 m. Der dokumentierte Gehäuseursprung liegt bei Z = 1,590 m: 130 mm Versatz. Die Gehäuseoberkante liegt bei Z = 1,621 m. Das ist eine akzeptierte Simulationsvereinfachung, keine deckungsgleiche Montage.

Navigation: Die IMU sitzt rechnerisch im Fahrzeugursprung; das ist weder ein bestätigter OEM-Einbauort noch ein vermessener Schwerpunkt. GNSS liegt am modellierten Gehäuse. 4G ist eine Kommunikationsanbindung und kein hier simulierter Umgebungssensor.

B Reproduzierbarkeit

Die folgende Anleitung beschreibt den Start beider Fahrzeuge Schritt für Schritt: vom Klonen des Repositorys über den Import in CARLA und das manuelle Fahren bis zu Sensoranzeigen, ROS-2-Anbindung und Slalom-Demo.

Quellcode, Parameterdateien und Prüfdaten liegen im Projekt-Repository (Rauch & Le, 2026). Die READMEs beschreiben Installation und Start. Die Messdatei des Motorprofils entstand vor der letzten X-Schwerpunktkorrektur; der spätere ROS-Fahrtest bestätigt den endgültigen Offset. Rohmessdaten werden nicht nachträglich an geänderte Sollwerte angepasst.

Lieferroboter in CARLA starten

Schritt-für-Schritt-Anleitung für Starship und Clevon: vom Repository über CARLA bis ROS 2

Stand 23.10.2026 · CARLA 0.10.0 · Unreal Engine 5.5.4 · ROS 2 Jazzy · Ubuntu 24.04 · Repository: git.ieem-ka.de/rari1014/last_mile-delivery-carla, Branch `clevon-starship-carla-0.10`

So liest du die Anleitung. Alles in den dunklen Kästen kannst du kopieren und im Terminal einfügen. Graue Zeilen mit `#` sind Kommentare. Die farbigen Schilder wie **TERMINAL 1** sagen dir, in welchem Terminal-Fenster ein Befehl läuft. Programme, die weiterlaufen sollen (CARLA, Fahrclient, Sensoren), brauchen jeweils **ein eigenes Terminal**.

1 Terminal-Grundlagen
2 Repository herunterladen
3 Einmalige Vorbereitung
4 Starship in CARLA importieren
5 CARLA starten und beenden
6 Starship: fahren und Sensoren
7 Starship über ROS 2
8 Clevon: fahren und Sensoren
9 Clevon über ROS 2
10 ROS-2-Demo: Slalom fahren
11 Probleme und Lösungen
12 Spickzettel

Das Prinzip in einem Bild

1 · CARLA
Der Simulator mit der Stadt Town10HD. Läuft die ganze Zeit.

2 · Fahrzeug
Ein Python-Programm setzt den Bot in die Stadt und lässt dich fahren.

3 · Sensoren
Weitere Programme bringen Kameras, Radar usw. an und zeigen die Daten.

4 · ROS 2
Daten und Steuerung für andere Programme bereitstellen.

1 Terminal-Grundlagen

Das Terminal ist ein Fenster, in das du Befehle tippst. Öffnen: `Strg`+`Alt`+`T`. Ein weiteres Terminal als Reiter im selben Fenster: `Strg`+`Umschalt`+`T`.

Befehl / Taste	Bedeutung
`pwd`	Zeigt, in welchem Ordner du gerade bist.
`ls`	Listet die Dateien und Ordner im aktuellen Ordner.
`mkdir ordner`	Legt einen neuen Ordner an.
`cd ordner`	Wechselt in einen Ordner. `cd ..` eine Ebene zurück, `cd ~` in deinen Home-Ordner.
`~`	Kurzform für deinen Home-Ordner, hier `/home/ieem`.
`Tab`	Vervollständigt Datei- und Ordnernamen. Spart Tippfehler.
`Pfeil hoch`	Holt den letzten Befehl zurück.
`Strg`+`Umschalt`+`V`	Einfügen im Terminal (`Strg`+`V` allein geht dort nicht).
`Strg`+`C`	Beendet das Programm, das gerade im Terminal läuft.

Lange Befehle mit `\` am Zeilenende gehören zusammen. Kopiere alle Zeilen auf einmal und füge sie ein.

2 Repository herunterladen

Das Repository enthält beide Bots: `01_Clevon1` und `02_starship_emodell`. Es ist öffentlich; eine Anmeldung ist nicht nötig.

2.1 Einmalig: Git und Git LFS

```
# Prüfen, ob Git und Git LFS da sind (Versionsnummer = installiert)
git --version
git lfs version

# Falls etwas fehlt (fragt nach deinem Passwort):
sudo apt install git git-lfs
git lfs install
```

2.2 Ordner anlegen und Repository klonen

```
# Ordner anlegen und hineinwechseln
mkdir -p ~/carla_projekt
cd ~/carla_projekt

# Repository mit beiden Bots holen (Modelldateien kommen über Git LFS mit)
git clone -b clevon-starship-carla-0.10 \
  https://git.ieem-ka.de/rari1014/last_mile-delivery-carla.git

cd last_mile-delivery-carla
git lfs pull
ls
```

Du solltest `01_Clevon1`, `02_starship_emodell` und `README.md` sehen. Alle weiteren Befehle gehen davon aus, dass das Repo unter `~/carla_projekt/last_mile-delivery-carla` liegt. Wer einen anderen Ordner wählt, ersetzt diesen Pfad überall.

```
# Später: neuesten Stand holen
cd ~/carla_projekt/last_mile-delivery-carla
git pull
git lfs pull
```

3 Einmalige Vorbereitung

> **Auf dem Institutsrechner ist das meiste bereits eingerichtet:** CARLA 0.10.0 unter `~/CarlaUE5`, Unreal Engine unter `~/UnrealEngine5_carla`, Python-Umgebung unter `~/CarlaUE5/.venv`, beide Fahrzeuge registriert, ROS 2 Jazzy installiert. Dort kannst du 3.1 und Kapitel 4 überspringen.

3.1 ROS 2 Jazzy installieren (nur falls nötig)

```
# Zeigt "jazzy" an -> fertig. Sonst installieren (dauert, fragt nach Passwort):
ls /opt/ros
bash ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2/install_ros_jazzy.sh
```

3.2 Python-Paket für den ROS-Ergänzungsknoten

```
~/CarlaUE5/.venv/bin/python -m pip install pyyaml
```

3.3 Steuernachricht `carla_msgs` bauen (je Fahrzeug einmal)

Damit ROS 2 Fahrbefehle an CARLA schicken kann. Ohne `sudo`, unter einer Minute.

```
bash ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2/ros_ws/build.sh
bash ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik/ROS2/native/ros_ws/build.sh
```

3.4 Schnelltest: Findet Python CARLA?

```
~/CarlaUE5/.venv/bin/python -c "import carla; print('CARLA-Python ok')"
```

4 Starship in CARLA importieren (nur auf neuem Rechner)

Ist `vehicle.starship.bot` in CARLA noch nicht vorhanden, baut ein Skript den Bot im laufenden Unreal Editor automatisch auf: Modell, Kollisionskörper, sechs Räder, Fahrzeug-Blueprint und Anmeldung bei CARLA.

1. Unreal Editor mit dem CARLA-Projekt starten (siehe unten) und warten, bis er offen ist.
2. Im Editor einmalig: *Edit → Project Settings → Plugins → Python → Enable Remote Execution* anhaken.
3. Import-Skript ausführen.

TERMINAL 1

```
# Editor starten (Ladezeit einige Minuten)
~/UnrealEngine5_carla/Engine/Binaries/Linux/UnrealEditor \
  ~/CarlaUE5/Unreal/CarlaUnreal/CarlaUnreal.uproject > /dev/null 2>&1 &

# Starship importieren und registrieren (ca. 5-7 min, endet mit SUCCESS)
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/scripts/unreal_editor/ue5
./build_all.sh

# Bordstein-Assistent anhängen (nur wenn das CARLA-Plugin die Komponente enthält)
python3 ue_remote.py s08_curb_assist_component.py

# Test im Editor: Play starten (beenden mit s07_stop.py)
python3 ue_remote.py s07_play.py
```

> **Bordstein-Assistent:** Die C++-Komponente liegt unter `02_starship_emodell/unreal/ue5_carla_plugin`. Sie muss einmalig in das CARLA-Plugin kopiert und der Editor neu gebaut werden (10–20 min). Ohne sie fährt der Bot normal, kommt aber nicht auf Bordsteine. Details: `02_starship_emodell/docs/IMPLEMENTIERUNG_CARLA_0.10.md`.

5 CARLA starten und beenden

CARLA ist der Simulator. Er muss laufen, bevor du einen Bot startest. Wähle **eine** der Varianten. Ein offener Unreal Editor (Kapitel 4) mit Play geht auch, verbraucht aber mehr Grafikspeicher.

Variante A: mit Fenster, ohne ROS 2

TERMINAL 1

```
~/UnrealEngine5_carla/Engine/Binaries/Linux/UnrealEditor \
  ~/CarlaUE5/Unreal/CarlaUnreal/CarlaUnreal.uproject \
  /Game/Carla/Maps/Town10HD_Opt -game -windowed \
  -ResX=1280 -ResY=720 -quality-level=Low -prefernvidia
```

Variante B: mit Fenster und ROS 2

TERMINAL 1

```
~/UnrealEngine5_carla/Engine/Binaries/Linux/UnrealEditor \
  ~/CarlaUE5/Unreal/CarlaUnreal/CarlaUnreal.uproject \
  /Game/Carla/Maps/Town10HD_Opt -game -windowed \
  -ResX=1280 -ResY=720 -quality-level=Low -prefernvidia --ros2
```

Variante C: ohne Fenster, mit ROS 2 (spart Grafikspeicher)

TERMINAL 1

```
~/UnrealEngine5_carla/Engine/Binaries/Linux/UnrealEditor \
  ~/CarlaUE5/Unreal/CarlaUnreal/CarlaUnreal.uproject \
  /Game/Carla/Maps/Town10HD_Opt -game -RenderOffScreen \
  -quality-level=Low -prefernvidia --ros2
```

> **Warten:** Der Start dauert 1–3 Minuten. **Wichtig:** `--ros2` mit **zwei** Bindestrichen, sonst bleibt ROS 2 aus. **Beenden:** im Terminal 1 `Strg`+`C`, vorher alle anderen Programme beenden.

```
# Läuft CARLA? (in einem neuen Terminal)
~/CarlaUE5/.venv/bin/python -c "import carla; c=carla.Client('localhost',2000); \
c.set_timeout(5); print('CARLA läuft:', c.get_world().get_map().name)"
```

6 Starship: fahren und Sensoren

Voraussetzung: CARLA läuft (Kapitel 5, Variante A oder B).

6.1 Bot spawnen und selbst fahren

TERMINAL 2

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik
~/CarlaUE5/.venv/bin/python manual_control_starship.py
```

Alternativ das unveränderte CARLA-Beispiel (ohne GNSS/IMU-Erweiterung):

```
cd ~/CarlaUE5/PythonAPI/examples
~/CarlaUE5/.venv/bin/python manual_control.py --filter 'vehicle.starship.bot'
```

Taste (im Fahrfenster)	Funktion
`W` / `S`	Gas / Bremse
`A` / `D`	Lenken links / rechts
`Q`	Rückwärtsgang ein/aus
`Leertaste`	Handbremse
`P` / `Tab` / `H`	Autopilot / Kameraposition / Hilfe
`Esc`	Beenden (Bot wird entfernt)

Der Starship fährt höchstens ca. 6 km/h. Bordsteine nimmt er senkrecht mit Vollgas; bleibt er hängen: kurz zurücksetzen und erneut anfahren.

6.2 Sensoranzeigen (je in einem eigenen Terminal)

TERMINAL 3

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik
~/CarlaUE5/.venv/bin/python starship_cameras.py        # 9 Kameras in Reitern
~/CarlaUE5/.venv/bin/python starship_tof.py           # 3 ToF-Tiefenkameras
~/CarlaUE5/.venv/bin/python starship_radars.py        # 4 Radare
~/CarlaUE5/.venv/bin/python starship_ultrasonic.py    # 8 Ultraschallsensoren
```

Immer nur **eine** Anzeige pro Terminal. Tasten: `1`…`9` / `Tab` Reiter, `S` Messdaten speichern, `Esc` schließen. Nicht mehrere Kamera- oder ToF-Fenster gleichzeitig öffnen (Grafikspeicher); bei Problemen `starship_cameras.py --preview`.

6.3 Kamera folgt dem Bot (nur mit CARLA-Fenster)

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik
~/CarlaUE5/.venv/bin/python starship_follow_cam.py       # --view top oder --view side
```

7 Starship über ROS 2

CARLA veröffentlicht die Sensordaten selbst über ROS 2. Voraussetzung: CARLA mit `--ros2` (Kapitel 5, Variante B oder C) und Kapitel 3.3 erledigt. Alle Topics liegen unter `/carla/starship/…`.

7.1 Bot spawnen: selbst fahren oder für ROS-Steuerung

TERMINAL 2

```
# selbst fahren:
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik
~/CarlaUE5/.venv/bin/python manual_control_starship.py

# ODER ohne Fahrclient (für 7.5); entfernen mit --remove
~/CarlaUE5/.venv/bin/python \
  ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2/spawn_starship.py
```

7.2 Sensoren für ROS 2 anbringen

TERMINAL 3

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2
~/CarlaUE5/.venv/bin/python starship_ros_sensors.py
# nur einzelne Gruppen: ... --groups nav radar   (nav, radar, tof, cameras)
```

7.3 Ergänzungsknoten: Ultraschall, Odometrie, Fahrzeuglage

TERMINAL 4

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2
source ros_ws/setup.bash
~/CarlaUE5/.venv/bin/python starship_ros_extras.py
```

7.4 Prüfen, ob Daten ankommen

TERMINAL 5

```
source /opt/ros/jazzy/setup.bash
ros2 topic list | grep starship                    # alle Starship-Topics
ros2 topic hz /carla/starship/starship_gnss        # Rate (ca. 5 Hz), Strg+C
ros2 topic echo --once /carla/starship/starship_gnss  # eine Nachricht
ros2 run tf2_ros tf2_echo map starship             # Position des Bots
```

7.5 Über ROS 2 steuern

Nur mit einem Bot aus `spawn_starship.py`; ein laufender Fahrclient überschreibt die ROS-Befehle.

TERMINAL 5

```
source ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2/ros_ws/setup.bash

# Geradeaus fahren (sendet 10x pro Sekunde, Strg+C beendet das Senden)
ros2 topic pub -r 10 /carla/starship/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl '{throttle: 0.7}'

# Linkskurve (steer: -1 = links, +1 = rechts)
ros2 topic pub -r 10 /carla/starship/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl '{throttle: 0.6, steer: -1.0}'

# Anhalten (einmalig senden, wartet auf die Verbindung)
ros2 topic pub --once -w 1 /carla/starship/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl '{brake: 1.0}'
```

> **Kein Not-Halt:** Hörst du mit dem Senden auf, fährt der Bot mit dem letzten Befehl weiter. Zum Anhalten immer ausdrücklich den Bremsbefehl senden.

8 Clevon: fahren und Sensoren

Voraussetzung: CARLA läuft (Kapitel 5, Variante A oder B).

8.1 Clevon spawnen und selbst fahren

TERMINAL 2

```
cd ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik
~/CarlaUE5/.venv/bin/python manual_control_clevon.py
```

Alternativ: `manual_control.py --filter 'vehicle.clevon.1'` im CARLA-Beispielordner. Tasten wie beim Starship, zusätzlich `L` Licht, `Umschalt`+`L` Fernlicht, `Z`/`X` Blinker. Der Clevon fährt bis ca. 50 km/h.

8.2 Sensoranzeigen (je in einem eigenen Terminal)

TERMINAL 3

```
cd ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik
~/CarlaUE5/.venv/bin/python clevon_cameras.py          # 6 Kameras, je 2 pro Reiter
~/CarlaUE5/.venv/bin/python clevon_radars.py           # 2 Front-Radare
~/CarlaUE5/.venv/bin/python clevon_lidar.py            # Dach-LiDAR
```

Kameras schonender: `clevon_cameras.py --low-load` oder `--preview`. Nachführkamera: `clevon_follow_cam.py`.

9 Clevon über ROS 2

Gleiches Vorgehen wie beim Starship. Voraussetzung: CARLA mit `--ros2`. Topics unter `/carla/clevon/…`.

TERMINAL 2

```
# selbst fahren ODER ohne Fahrclient spawnen (für ROS-Steuerung):
cd ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik
~/CarlaUE5/.venv/bin/python manual_control_clevon.py
~/CarlaUE5/.venv/bin/python ROS2/native/spawn_clevon.py
```

TERMINAL 3

```
cd ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik/ROS2/native
~/CarlaUE5/.venv/bin/python clevon_ros_sensors.py --preview
# Gruppen: nav radar lidar cameras;  Kameras: --preview / --low-load / ohne (Full HD)
```

TERMINAL 4

```
cd ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik/ROS2/native
source ros_ws/setup.bash
~/CarlaUE5/.venv/bin/python clevon_ros_extras.py
```

TERMINAL 5

```
source ~/carla_projekt/last_mile-delivery-carla/01_Clevon1/Sensorik/ROS2/native/ros_ws/setup.bash
ros2 topic list | grep clevon
ros2 topic pub -r 10 /carla/clevon/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl '{throttle: 0.5}'
ros2 topic pub --once -w 1 /carla/clevon/vehicle_control_cmd \
  carla_msgs/msg/CarlaEgoVehicleControl '{brake: 1.0}'
```

> Für den Clevon gibt es zusätzlich eine ältere ROS-2-Bridge (Windows/WSL, ROS 2 Humble) in `01_Clevon1/Sensorik/ROS2` mit Licht- und Klappenbefehlen. Nicht gleichzeitig mit dieser Anleitung betreiben.

10 ROS-2-Demo: Slalom fahren (beide Bots)

Für Vorführungen: Ein Skript setzt den Bot an eine lange gerade Straße, lenkt ihn über ROS 2 im Slalom (Topic `vehicle_control_cmd`) und zeigt zwei **Fahrzeugkameras aus den JSON-Sensorprofilen**, deren Bilder es selbst über ROS 2 empfängt: beim Starship Frontkamera K1 und Heckkamera K8, beim Clevon Front- und Heckkamera. Position, Sichtfeld und Rate entsprechen dem Profil. Über jedem Bild steht das ROS-Topic. Am Ende der Geraden setzt das Skript den Bot an den Start zurück; die Kamera im CARLA-Fenster folgt ihm.

Voraussetzung: CARLA mit `--ros2` und Fenster (Kapitel 5, Variante B) und Kapitel 3.3. Kein anderer Fahrclient für diesen Bot darf laufen.

> **Auch für den Clevon im Starship-Ordner bleiben!** Das Demo-Skript und sein `ros_ws` liegen nur unter `02_starship_emodell/Sensorik/ROS2`. Das Skript steuert beide Fahrzeuge; den Clevon wählst du mit `--vehicle clevon`. Im Ordner `01_Clevon1/Sensorik/ROS2` gibt es weder `ros2_slalom_demo.py` noch `ros_ws/build.sh` (Fehler *No such file or directory*).

TERMINAL 2

```
# immer im Starship-Ordner, auch für den Clevon
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2
bash ros_ws/build.sh          # nur beim ersten Mal nach dem Klonen
source ros_ws/setup.bash

# Starship (ca. 5 km/h):
~/CarlaUE5/.venv/bin/python ros2_slalom_demo.py

# ODER Clevon (ca. 15 km/h):
~/CarlaUE5/.venv/bin/python ros2_slalom_demo.py --vehicle clevon
```

Option	Wirkung
`--cameras ROLLE ROLLE`	zwei andere Kameras aus dem Profil, z. B. `clevon_camera_left_upper` `clevon_camera_right_upper`
`--duration 60`	nach 60 s automatisch beenden
`--spawn 12`	festen Spawnpunkt statt automatischer Suche
`--no-view`	ohne Kamerafenster

Beenden: `Esc` im Demo-Fenster oder `Strg`+`C`. Das Skript sendet dann einen Bremsbefehl und entfernt Bot und Kameras. Zusätzliche Sensoren und Topics wie in Kapitel 7 lassen sich parallel in weiteren Terminals starten:

TERMINAL 3

```
cd ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2
~/CarlaUE5/.venv/bin/python starship_ros_sensors.py --groups nav radar tof
```

TERMINAL 4

```
source ~/carla_projekt/last_mile-delivery-carla/02_starship_emodell/Sensorik/ROS2/ros_ws/setup.bash
ros2 topic list | grep starship
ros2 topic hz /carla/starship/starship_radar_front/point_cloud
ros2 topic echo /carla/starship/vehicle_control_cmd
```

11 Probleme und Lösungen

Meldung / Verhalten	Lösung
`No such file or directory`	Falscher Ordner oder Tippfehler. Mit `pwd` und `ls` prüfen, die `cd` -Zeile erneut einfügen.
`ros2_slalom_demo.py` oder `ros_ws/build.sh` nicht gefunden (Clevon)	Die Demo liegt nur im Starship-Ordner: `cd …/02_starship_emodell/Sensorik/ROS2` , dann `--vehicle clevon` .
`time-out ... while waiting for the simulator`	CARLA läuft noch nicht oder lädt noch. Warten, Test aus Kapitel 5.
`get_trafficmanager ... std::exception`	Ein alter Fahrclient läuft noch: alle Terminals mit `Strg` + `C` beenden oder `pkill -f manual_control` .
`Out of memory on Vulkan` , CARLA stürzt ab	Grafikspeicher voll. Weniger Kameras, `--preview` , Variante C. Kein zweites CARLA und keinen Editor parallel.
Keine ROS-Topics	CARLA ohne `--ros2` gestartet oder `source …/setup.bash` vergessen. Im CARLA-Log muss „ROS2: Enabling ROS2“ stehen.
`Package 'carla_msgs' not found`	Kapitel 3.3 ausführen und vor dem Befehl `source …/ros_ws/setup.bash` .
Bot reagiert nicht auf ROS-Befehle	Läuft noch ein Fahrclient? Bot mit `spawn_starship.py` bzw. `spawn_clevon.py` neu setzen.
`No module named 'carla'` oder `'yaml'`	Immer `~/CarlaUE5/.venv/bin/python` verwenden; für `yaml` Kapitel 3.2.
`NO_EDITOR_FOUND` beim Import	Editor noch nicht fertig geladen oder Remote Execution nicht aktiviert (Kapitel 4).
`vehicle.starship.bot` nicht gefunden	Starship noch nicht importiert: Kapitel 4.
Alles hängt	Alle Programme beenden, notfalls `pkill -f UnrealEditor` und CARLA neu starten.

Reihenfolge beim Beenden

1. Sensoranzeigen und ROS-Programme: `Esc` bzw. `Strg` + `C` (Sensoren werden entfernt).
2. Fahrclient: `Esc` .
3. CARLA: im Terminal 1 `Strg` + `C` .

12 Spickzettel

Starship

Fahrzeug-ID	`vehicle.starship.bot`
Ordner	`02_starship_emodell/Sensorik`
Fahren	`manual_control_starship.py`
Anzeigen	`starship_cameras/_tof/_radars/_ultrasonic.py`
ROS	`ROS2/spawn_starship.py` , `starship_ros_sensors.py` , `starship_ros_extras.py`
Demo	`ROS2/ros2_slalom_demo.py`
Topics	`/carla/starship/…`
Sensoren	9 Kameras, 3 ToF, 4 Radar, 8 Ultraschall, GNSS, IMU

Clevon

Fahrzeug-ID	`vehicle.clevon.1`
Ordner	`01_Clevon1/Sensorik`
Fahren	`manual_control_clevon.py`
Anzeigen	`clevon_cameras/_radars/_lidar.py`
ROS	`ROS2/native/spawn_clevon.py` , `clevon_ros_sensors.py` , `clevon_ros_extras.py`
Demo	im Starship-Ordner: `02_starship_emodell/Sensorik/ROS2/ros2_slalom_demo.py --vehicle clevon`
Topics	`/carla/clevon/…`
Sensoren	6 Kameras, 2 Radar, LiDAR, GNSS, IMU

Typische Sitzung (ROS-2-Demo)

Terminal	Was läuft
TERMINAL 1	CARLA mit `--ros2` und Fenster (Kapitel 5 B)
TERMINAL 2	`ros2_slalom_demo.py` (Kapitel 10)
TERMINAL 3	Sensoren für ROS (`*_ros_sensors.py`)
TERMINAL 4	Prüfen mit `ros2 topic …`

Weiterführend im Repo: `README.md` , `02_starship_emodell/Sensorik/ROS2/README.md` , `01_Clevon1/Sensorik/ROS2/native/README.md` , `02_starship_emodell/docs/IMPLEMENTIERUNG_CARLA_0.10.md` .

von KI erstellt